

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 12:27:12
Файлов исходного кода: 83
Суммарный объём кода: 1.17 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/83: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

` .github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow dispatch:

```
 path: ./dist
```

```
 deploy:
```

```
 name: Publish to GitHub Pages
```

```
 needs: build
```

```
 runs-on: ubuntu-latest
```

```
 environment:
```

```
 name: github-pages
```

```
 url: ${ steps.deployment.outputs.page_url }
```

```
 steps:
```

```
 - name: Deploy
```

```
 id: deployment
```

```
 uses: actions/deploy-pages@v5
```

```
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
```

```
new file mode 100644
```

```
index 00000000..17418e3
```

```
--- /dev/null
```

```
+++ b/.github/workflows/deploy-pages.yml
```

```
@@ -0,0 +1,60 @@
```

```
+name: Deploy app to GitHub Pages
```

```
+
```

```
+on:
```

```
+  push:
```

```
+    branches:
```

```
+      - main
```

```
+      # Позволяет проверить Pages прямо из рабочей ветки
```

```
+      - "arena/**"
```

```
+  workflow_dispatch:
```

```
+
```

```
+permissions:
```

```
+  contents: read
```

```
+  pages: write
```

```
+  id-token: write
```

Универсальное пособие • полный исходный код

```
-----
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+....
```

`.github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
|→ page-0001.png ... page-NNNN.png (
|
|→ full_code_all_page
#
```

# Готовые TXT и PDF коммитятся обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

- ```
for policy in /etc/ImageMagick-6/policy.xml /
  if [ -f "$policy" ]; then
    sudo sed -i 's/rights="none" pattern="\{P
  fi
done
convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| PNG-страниц | \$(ls tmp/export-pages) |"
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
      -   node-version: "22"
      +   node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....
```

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { chapters } from '../data/content';
import { Navbar } from '../components/Navbar';
import { Sidebar } from '../components/Sidebar';
import { MobileToc } from '../components/MobileToc';
import { ChapterView } from '../components/ChapterView';
```



```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" style={{ padding: '10px 0' }}>
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" style={{ padding: '10px 0' }}>
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" style={{ padding: '10px 0' }}>
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" style={{ padding: '10px 0' }}>
 <div className="min-w-0">
 <div className="text-[10px] uppercase" style={{ padding: '2px 0' }}>
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" style={{ padding: '2px 0' }}>
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" style={{ padding: '10px 0' }}>
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
 style={{ width: `${progress}%` }}
 />
 </div>
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

-----  
````  

Patch относительно `main`

````diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";

import { SimulatorHub } from "../components/SimulatorHub";

+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

const [activeChapterId, setActiveChapterId] = useState<string>();

const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

- ) : (

+ ) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+ ) : (

+ <PythonCompiler />

}}

```

 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-mono">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-projects-btn"
 onClick={() => setActiveTab('projects')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'projects'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Проекты
 </button>
 </div>
</div>

```

```

 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
</div>
</header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
    >

```

Универсальное пособие • полный исходный код

import { CheckCircle2, ExternalLink, Info, Loader2, Play

```
const PYODIDE_VERSION = "0.27.7";  
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo  
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
```

```
const STARTER_CODE = `# Первый запуск загрузит Python в  
name = "алгоритмы"
```

```
for number in range(1, 4):  
    print(f"{number}. Учим {name}!")  
`;  
;
```

```
type PyodideRuntime = {  
  runPythonAsync: (source: string) => Promise<unknown>;  
  setStdout: (options: { batched: (message: string) => ;  
  setStderr: (options: { batched: (message: string) => ;  
  setStdin: (options: { stdin: () => string | number | ;  
};
```

```
type PyodideModule = {  
  loadPyodide: (options: { indexURL: string }) => Promi  
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = nu
```

```
function getPythonRuntime() {  
  if (!runtimePromise) {  
    runtimePromise = import(/* @vite-ignore */ PYODIDE_I  
      (module as PyodideModule).loadPyodide({ indexURL:  
    });  
  }  
  return runtimePromise;  
}
```

```
function errorText(error: unknown) {  
  if (error instanceof Error) return error.message;  
  return String(error);  
}
```

```

        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:p
    <div className="max-w-6xl mx-auto">
        <div className="flex flex-col sm:flex-row sm:it
            <div>
                <div className="inline-flex items-center gap
                    <Terminal className="w-4 h-4" /> Боковая
                </div>
                <h2 className="text-2xl sm:text-3xl font-ex
                <p className="text-slate-400 mt-2 max-w-2xl
                    Пишите небольшой код и запускайте его пря
                    который переводит CPython в WebAssembly.
                </p>
            </div>
            <a
                href="https://pyodide.org/"
                target="_blank"
                rel="noreferrer"
                className="inline-flex items-center gap-1.5
            >
                Как это работает <ExternalLink className="w
            </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr)
            <section className="rounded-2xl border border
                <div className="flex flex-wrap items-center
                    <div className="flex items-center gap-2">

```

```

        -none focus:ring-2 focus:ring-inset focus:ring-slate-500
        placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
        <label className="block px-4 pt-3 text-slate-500" >
            Ввод для input() • по одной строке на клавишу
        </label>
        <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border border-slate-800"
        />
    </div>
</section>

<section className="rounded-2xl border border-slate-800" >
    <div className="flex items-center justify-between" >
        <div className="flex items-center gap-2" >
            <Terminal className="w-4 h-4 text-emerald-500" />
        </div>
        <span className={`inline-flex items-center gap-2`} >
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
            {runtimeReady ? "готов" : "не загружен"}
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px] px-4 py-4 leading-6 text-slate-300" >
        {output}
    </pre>
    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
    </div>
</section>
</div>

```

```
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+});
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
```



```

+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:p
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:i
+       <div>
+         <div className="inline-flex items-center g
+           <Terminal className="w-4 h-4" /> Боковая
+         </div>
+         <h2 className="text-2xl sm:text-3xl font-e
+         <p className="text-slate-400 mt-2 max-w-2x
+           Пишите небольшой код и запускайте его пр
+           который переводит CPython в WebAssembly.
+         </p>
+       </div>
+       <a
+         href="https://pyodide.org/"
+         target="_blank"
+         rel="noreferrer"
+         className="inline-flex items-center gap-1.5
+       >
+         Как это работает <ExternalLink className="
+       </a>
+     </div>
+
+     <div className="grid xl:grid-cols-[minmax(0,1.
+       <section className="rounded-2xl border borde
+       <div className="flex flex-wrap items-cente
+         <div className="flex items-center gap-2"
+           <span className="w-2.5 h-2.5 rounded-fr
+           <span className="text-sm font-bold tex
+           <span className="text-[11px] text-slate
+         </div>
+         <div className="flex items-center gap-2"
+           <button
+             type="button"

```

```

+         </label>
+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.target.value)}
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-transparent border-slate-200 p-2"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border border-slate-200 p-4" >
+         <div className="flex items-center justify-between" >
+             <div className="flex items-center gap-2" >
+                 <Terminal className="w-4 h-4 text-emerald-500" />
+             </div>
+             <span className={`inline-flex items-center gap-2`} >
+                 <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+                 {runtimeReady ? "готов" : "не загружен"}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" >
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4" >
+         </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" >
+     <div className="flex gap-2 rounded-xl border border-slate-200 p-4" >
+         <Info className="w-4 h-4 shrink-0 text-emerald-500" />
+         <span>Ctrl или Cmd + Enter запускает код. l
+     </div>
+     <div className="flex gap-2 rounded-xl border

```

```

    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});

```

Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {

```

## Универсальное пособие • полный исходный код

- Определение/правило в центре (`border-blue-500` или
  - Обязательная сетка аналогий (2 колонки: неформальн
  - "Душные" детали (код, формулы, сложные доказательства
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы в файлах `src/data/content.ts`. Не используй чистый markd

ФАЙЛ 4/83: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen" >
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/83: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам",
 "requestFramePermissions": [],
 "majorCapabilities": [
 "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
]
}
```

## Универсальное пособие • полный исходный код

```

 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/83: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.5 КБ

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5174  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

| Команда              | Шаг       | Результат            |
|----------------------|-----------|----------------------|
| ---                  | ---       | ---                  |
| `npm run export:txt` | код → txt | `public/export/full` |
| `npm run export:png` | txt → png | `tmp/export-pages/`  |

## Универсальное пособие • полный исходный код

- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы а известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается кар мини-анимацией и кнопкой «Показать симулятор» (симуля Словарь – `src/data/glossary.ts`, мини-анимации – `src
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

### ## Структура

...

|                    |                                  |
|--------------------|----------------------------------|
| src/               |                                  |
| App.tsx            | – оболочка, привязка глав к визу |
| components/        | – интерактивные симуляторы (Дейк |
| data/              | – контент пособия (главы/билеты  |
| scripts/export/    | – пайплайн код → txt → png → pdf |
| public/export/     | – сгенерированные TXT и PDF (обн |
| .github/workflows/ | – автоматизация                  |
| ...                |                                  |

---

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен азуемые теги и классы.

### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика раграфы `

`.

### ## Математические формулы

В приложении используется MathJax.

## Универсальное пособие • полный исходный код

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

|                    |           |                      |
|--------------------|-----------|----------------------|
| Глава              | id        | Что делать           |
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

|               |                                |                                        |
|---------------|--------------------------------|----------------------------------------|
| Билет         | Тема                           | Статус                                 |
| ---           | ---                            | ---                                    |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Контент вхолостую.          |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; аналогии**.        |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть без аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть без аналогии. |

Дополнительно — «осиротевшие» визуализаторы без глав (контент вхолостую)

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

|       |     |             |
|-------|-----|-------------|
| Глава | id  | Комментарий |
| ---   | --- | ---         |

## Универсальное пособие • полный исходный код

|    |                                      |   |   |   |
|----|--------------------------------------|---|---|---|
| 16 | 16. Графы. Кратчайший путь. Флойд    | - | - | - |
| 17 | 17. Графы. Алгоритм Джонсона         | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 | - | - |
| 21 | 21. Префикс-функция (п), КМП         | 4 | - | - |
| 22 | 22. Z-функция                        | 4 | - | - |
| 23 | 23. Алгоритм Ахо-Корасик             | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы       | - | - | - |
| -  | Алгоритмы на карте                   | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                   | - | - | - |
| -  | Мосты в графах: код + визуализация   | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be``.  
это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/83: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
```



```

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

=====

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

-----

ФАЙЛ 11/83: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 КБ

-----

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
```

```

const progress = useMemo(() => ((index + 1) / chapter

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded-t" />
 </main>
 </div>
 </>
) : (
 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded-t" />
 </main>
 </div>
)}
 </Navbar>
 </div>
)

```

}

ФАЙЛ 12/83: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

const escapeRe = (s: string) => s.replace(/[\.*+?^${}()|]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N}_-])(${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на
```

```

 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("div"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("div"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {

```

```
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
```

```
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
```

```
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```

---

ФАЙЛ 14/83: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/83: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
```

```

import { Chapter } from "../types";

export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга по 1 см)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```



```
<p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 двоичная куча. Пара – точка на декартовой
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
 платит $\approx n \cdot \log n$ сравнений, counting по ма
 точка вставляется спуском от корня: $x \leq \text{уз}$
 единственно. Весь процесс – в симуляторе под
 «соединить сам» – кликаешь две точки,
 ь родитель», «получился бы цикл», «равные »,
 «Миф $2k/2k+1$ » и «Поиск $\neq$ сортировка»
 ждый запрещённый ход.</p>
```

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">

 ext-2xl text-slate-300">Й
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">Sp
 х»». Спуск от корня: узел, который вместе
 скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
```

Аналогия 2: Merge – стыковка

```

 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}
}

```

</div>  
 </details>

<details class="bg-slate-900/40 rounded-lg p-4 font-bold text-slate-300 border-slate-700/50">

Код: вставка, merge, erase (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300">  
 <pre class="bg-slate-950 p-4 rounded

```

def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — в левое
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

```

```

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)

```

<pre class="bg-slate-950 p-4 rounded

```

def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

```

```

def erase(root, key): # удалить произвольный элемент

```

не трогаешь — он уполз вглубь. Ничего не у  
ине, а холодное само тонет.</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg  
<div class="absolute -top-3 left-4  
border-rose-500 shadow-md">

Аналогия 2: Тропинка через га.

</div>

<p class="text-slate-300 text-sm">Ч  
дерево временно разрешено растить кривым, з  
да живёт в амортизированном  $O(\log n)$ .</p>

</div>

</div>

<p class="text-slate-300 text-sm mt-6">Почем  
от «двух зигов подряд» — а формы-то разные.  
от — отдельный шаг (прицел на ребро → повор  
ig-Zig и Zig-Zag видно целиком: порядок пов

<div class="bg-slate-900/40 rounded-lg bord

<p class="text-slate-300 text-sm mb-2">

<p class="text-slate-400 text-sm mb-1">

ое; два нижних зига оставили бы почти ту же

<p class="text-slate-400 text-sm mb-1">

а к верху, симулятор подсвечивает переехавш

<p class="text-slate-400 text-sm">• «Sp  
о амортизированная.</p>

</div>

<p class="text-slate-300 text-sm mt-4"><b c  
ое <code class="bg-slate-950 px-2 py-0.5 ro  
одиночная операция — до <code class="bg-sla  
> $O(n)$ </code>. Все операции через один splay  
ое) → вырезать корень → merge двух половин

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate

```

 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоугольника
 </div>
 <p class="text-slate-300 text-sm mb-4">
 з него маленький целевой прямоугольник в центре.
 ый верхний угол отрезается дважды! Поэтому формула
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `

```

```

<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. А
```

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и $f_{up}[v] > tin[u]$.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
order border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
этот мост, экономикой обоих кусков придется
 </div>
 </div>
 </div>
 </div>
</section>`,
},
{
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```

```

 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрыв
 </div>
 <p class="text-slate-300 text-sm mb
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-выш
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="bg-slate-900 p-4 rounded-lg mb-r
 <p class="text-lg text-blue-300 italic m
 <p class="text-xl font-mono text-white">
 d-длина.</p>
 </div>
 </div>
 </div>
`
 }
]

```

```

 order border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-6">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Основное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Основные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 order border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>

```



```

 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 i]), который одновременно является её суффиксом.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс и суффикс)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 идишь "АБРА". Если ты ошибешься при поиске
 концовка "АБРА" совпадает с началом "АБРА"
 мя перепроверок!</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```

```

 <p class="text-xl font-mono text-white">
щегося с i.</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной коп
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",

```

```
 </div>
</section>`,
 },
];
```

---

ФАЙЛ 18/83: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Не бы
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
```

```

 ng> ровно раз и вернуться домой.
 Ты обходишь все бары (вершины) ровно
 Главное – зайти и выйти, не заходя
 Никто не знает, как решать быстро.

 лноту).
 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Душный режим: Формальное доказательство
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400">Теорема Эйлера
 <p>
 Связный граф содержит эйлеров цикл ⇔

Эйлеров путь (не цикл) ⇔ ровно 0 или 2
 </p>
 <p class="text-rose-400">Почему Гамильтон?
 <p>
 Задача коммивояжёра (TSP) – это взрыв мозга.
 Доказано, что любой полиномиальный алгоритм
 В общем, не парься. Просто запомни это.
 </p>
 </div>
 </details>
</div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 type: "html",
 content: `<section id="bridges-code" class="mb-20 scrollable">
 <div class="flex items-center mb-6">

```

```

 </p>
 </div>

```

```

<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">

```

Аналогия: Кровеносный сосуд

```
 </div>

```

```
 <p class="text-slate-300 text-sm mb-4">

```

Граф – это кровеносная система. Ребра (артерии и вены) – это не мост, живём.

Если же перерезать единственную артерию, то наступит катастрофа.

Алгоритм DFS ищет такие «критические» ребра.

```
 </p>

```

```
 </div>

```

```
</div>

```

```

<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Полный код на C++ (Скрыто)
 </summary>

```

Полный код на C++ (Скрыто)

```
 </summary>

```

```

 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Классическая реализация – <code class="text-slate-400">
 оходов.</p>

```

```

 <div class="bg-slate-950 p-4 rounded-lg border-rose-500 shadow-md">

```

```
 <pre class="text-xs font-mono text-slate-400">

```

```
#include <vector>;
```

```
#include <algorithm>;
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежности
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```

 </div>
</section>`,
 },
 {
 id: "planarity-euler-formula",
 title: "Планарность: K5, K3,3 и формула Эйлера",
 type: "html",
 content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic m
 <p class="text-3xl font-mono text-white
 <div class="text-sm text-slate-400 spac
 <p><span class="text-blue-400 font-l
 <p><span class="text-emerald-400 fo
 <p><span class="text-rose-400 font-l
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
 </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 order-green-500 shadow-md">

```

```


Каждая грань ограничена минимум

Сумма длин граней = $2E = 20$.

Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ — <sp

Значит, K_5 не может быть планарн
 </p>
 </div>
</details>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 content: `<section id="graph-articulation" class="ml
<div class="flex items-center mb-6">
 <span class="bg-rose-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Точки
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 r

 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-rose-300 italic r
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">

```

<p>

Для стартовой вершины обхода правил  
то выше неё прыгать некуда). Поэтому для не  
у него <strong>больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}
```

</div>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl bor

<h4 class="text-lg font-bold text-indigo-400

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориент

</p>

<ul class="list-disc list-inside text-sm te

<li><strong class="text-rose-400">Точки



```

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
 if (!toRemove.includes(c.id)) {
 // We prefer the newer versions (like tickets14_20
 dedup.set(c.id, c);
 }
});

// Add the new ones
if (newChapters) {
 newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
 const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
 return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi
 "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
 "string-z-func", "segment-trees", "sparse-table", "dynam
];

```

```

 demo: "bfs",
 simulator: "queue-bfs",
},
{
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину"],
 title: "DFS – обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся – откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
},
{
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта – шутлив
 quicksort.",
 formal:
 "Поиск одного элемента в отсортированном массиве -
 O(n log n). Бинарный поиск сортировку не выполн
 complexity: "поиск: O(log n); сортировкой не являет
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
 formal:
 "Полное бинарное дерево в массиве: дети узла i ле
 complexity: "вставка/извлечение O(log n), «посмотре
 demo: "heap",

```

```

 formal: "Из корня по символам; в вершине хранится ф.
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то об
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксн
 о есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень
 кладём с в очередь.",
 complexity: " $O(\text{суммарной длины словаря} \times \text{алфавит})$ ",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошл
 строки.",
 formal: " $\text{link}(v) = \text{go}(\text{link}(\text{parent}(v)), \text{ch})$. Считае
 demo: "suffix-link",

```

```

 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sp",
 "разреженная", "двоичные подьёмы", "двоичный подь",
],
 title: "Разреженная таблица (метод двоичных подьёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1,
 м.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь ма
 товых кусков.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается",
 formal: "Ребро (u,v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где $\text{tin}[u]$ — время захода в вершину u при обходе в глубину (и $\text{low}[v]$ — минимальное tin среди всех вершин, достижимых из v, не включая само ребро в родителя — нет).",
 complexity: " $O(V + E)$ одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{

```



```

 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из А в В. «Дешёвый» — по с
 formal:
 "Дейкстра — неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»
 formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "са
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас
 formal:
 "Корректность обычно доказывается «леммой о разре
 demo: "mst",

```

```
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "список смежности"],
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быстрая и экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Список:",
},
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкстры"],
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший путь к цели.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали минимальную кучу.",
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллмана"],
 title: "Алгоритм Форда–Беллмана",
 short:
 "Тупой, но честный: $V-1$ раз проходим по ВСЕМ рёбрам.",
 formal:
 "После k -й итерации гарантированно верны все кратчайшие пути, если их нет, то в графе есть отрицательный цикл.",
 complexity: " $O(V \cdot E)$ ",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "floyd",
```

```

 simulator: "mst-boruvka",
 },
 {
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч
 formal:
 "Сортировка $O(E \log E)$ + E операций union/find. C
 complexity: " $O(E \log E)$ ",
 demo: "mst",
 simulator: "mst-kruskal",
 },
 {
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мост
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая
 complexity: " $O(V + E)$ ",
 demo: "components",
 simulator: "graph-dfs-bfs",
 },
 {
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:

```



```

 formal:
 "Без балансировки дерево может вырождаться в список",
 complexity: "O(высоты): от O(log n) до O(n)",
 demo: "segment-tree",
 },
 {
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём большой и маленький, а лишние вычитаем.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
 complexity: "запрос O(1) после O(nm) предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведённый"],
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно свести к B, то B можно решить.",
 formal:
 "A ≤p B: есть полиномиальная функция, переводящая экземпляры A в экземпляры B, решая которые можно решить A.",
 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива.",
 }
}

```

```

 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию",
 demo: "zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в од
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
 demo: "zig-zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева",
 demo: "zig-zag",
 simulator: "splay-rotations",
 },
];

```

/\*\* Быстрый доступ по id. \*/

export const GLOSSARY\_BY\_ID = new Map(GLOSSARY.map((e) => {

/\*\* Все метки, отсортированные от длинных к коротким (ч

export const GLOSSARY\_LABELS: { label: string; id: string

e.labels.map((label) => ({ label, id: e.id })))

).sort((a, b) => b.label.length - a.label.length);

```

 <h2 class="text-3xl font-bold text-white">Дейкс
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra">
 </div>
```

```
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 охождение улицы), Дейкстра сломается, так как
 </div>
 </div>
```

```
<details class="bg-slate-900/40 rounded-lg border"
 <summary class="p-4 font-bold text-slate-300">
```

```

 border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman" class="w-100">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-4">
 г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
</div>
</section>`
 },
 {
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">

```

```
category: "Строки",
content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">Осн
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы движемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 l
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
```

```

nodes[child].link = nodes[nodes[v].link].go

// Если суффиксная ссылка сама является сло
// Иначе наследуем terminal_link
if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
} else {
 nodes[child].term_link = nodes[nodes[ch
}

q.push(child);
 }
}
}

</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-purple-400
 <p class="text-slate-300 text-sm mb-4">Когда
 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>

```

```

 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий
ого критерия. Это NP-полная задача, которую не
 },
 {
 question: "Куда ведёт эйлеров путь, если нечётных
options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет – нужно удалять лишние рёбра",
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлер
ровно 2 нечётные или 0."
 }
],
"bridges-code": [
 {
 question: "После DFS у нас $low[u] = 4$, а $tin[v] =$
options: [
 "Да, так как $low[u] (4) > tin[v] (2)$ ",
 "Нет, $low[u]$ должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: $low[u] > tin[v]$. В н
 },
 {
 question: "Для чего нужен массив $low[v]$?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: " $low[v]$ хранит минимальное время вхо
поиска мостов и точек сочленения."
 },

```

```
 "Потому что 5 вершин — несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, 3V - 6 = 9. 10 > 9"
 }
]
};
```

---

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

---

ФАЙЛ 23/83: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../../../types';

export const tickets1to5: Chapter[] = [
 {
 id: "segment-trees",
 title: "1. Дерево отрезков с операциями на отрезках",
 type: "html",
 description: "Дерево отрезков — структура данных для",
 category: "Оптимизации и Продвинутое",
 content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-violet-700">
 </div>

 <div class="space-y-8">
 <div id="intro" class="bg-slate-700/50 p-6 rounded">
 <h3 class="text-xl font-bold text-blue-400">
 </div>
 </div>
`
 }
];
```



```

 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}
</pre>
</div>
</details>
</div>
</div>
</section>
`
},
{
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Сложность
 <h2 class="text-2xl sm:text-3xl font-bold text-white">2. Двумерная разреженная матрица (Sparse Table)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Задача
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Дана матрица ST размера $L \times R$ и k
 <p class="text-xl font-mono text-white">Найти сумму элементов на отрезке $[l, r]$ по строке i и столбцу j .
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">

```

```
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 по значению X. Все, кто меньше X улетают в левую часть.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вправо). Мы просто смотрим на корни: у кого Y больше, тот становится
 вого). Мы просто смотрим на корни: у кого Y
 как потомок.</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-300">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
```

```

 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb
вится корнем дерева (VIP-статус). Если ты ч
ка почти не требуется времени!</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Балансировка чере
 </div>
 <p class="text-slate-300 text-sm mb
и становиться кривым как бамбук. Но как тол
 </p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounde
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;
 x.right = p;
 } else { // Left rotate
 p.right = x.left;
 x.left = p;
 }
}

```

```

 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb-4">
 з него маленький целевой прямоугольник в цел
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
};

```

```

 border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-1">
 дный и не умеет пересматривать уже "зафиксир
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-p-6">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-rose-400">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-400
 order border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-1">
 еряет ВСЕ возможные дороги V-1 раз подряд.

```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг <code class="bg-slate-900 text-pink-400">k</code>
 шу себе проходить через вершину <code class="bg-slate-900 text-pink-400">k?</code>
 </p>
 <ul class="text-sm text-slate-300 space-y-4">
 Шаг k=0: Ты знаешь только
 Шаг k=1: Разрешаем пересадки
 е> через путь <code class="text-indigo-300">k</code>
 Шаг k=2: Разрешаем пересадки
 нутри этих путей уже могут быть пересадки ч
 ...
 Шаг k=N: Ты проверил все

 </div>
</div>
</section>`
 },
 {
 id: "johnson-algo",
 title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 </div>
 </div>
 </div>
`
 }
]

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-emerald-300 italic"
 <p class="text-xl font-mono text-white"
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb"
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
},
{
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"

```

```

<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру племена
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
</div>
</section>`
 }
];

```

ФАЙЛ 25/83: src/data/tickets/ticket21\_24.ts  
 КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМЕР: 1.2 КБ

```

import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Префикс-функция (π) — Взгляд назад (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Строки
 <h2 class="text-2xl sm:text-3xl font-bold text-white">21. Префикс-функция (π) — Взгляд назад (КМП)</h2>
 </div>
 <div class="space-y-8">

```



```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<p>Тот же пример: abcabcd</p>
```

```
<ul class="list-disc pl-5 space-y-2
```

```
a: Префиксов нет. <code>
```

```
ab: Из начала ("a")
```

```
>
```

```
abc: Опять мимо. <code>
```

```
abca: 0! Начало ("a"
```

```
</code>
```

```
abcab: Начало ("ab"
```

```
/li>
```

```
abcabc: Начало ("abc"
```

```
e>
```

```
abcabcd: "d" всё исп
```

```
"abca" vs "abcd"). Конец "abcd" не совпада
```

```

```

```
</div>
```

```
</details>
```

```
<details class="bg-slate-900/40 rounded-lg
```

```
<summary class="p-4 font-bold text-slate-300
```

```
-b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounded
```

```
vector<int> pi(s.length());
```

```
for (int i = 1; i < s.length(); i++) {
```

```
 int j = pi[i-1];
```

```
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
```

```
 if (s[i] == s[j]) j++;
```

```
 pi[i] = j;
```

```
}</pre>
```

```
</div>
```

```

 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопиаст
 </div>
 <p class="text-slate-300 text-sm mb"
 В LLM Z-функция может применять
[1]</code> (где <i>i</i> указывает назад на
сама себя.
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Пример вычисления (Скрыто)
</summary>
 <div class="p-5 text-sm text-slate-300"
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2"
 <code>Z[0]</code> – не счита
 <code>Z[1]</code> (bcabcd
[1] = 0</code>
 <code>Z[2]</code> (cabcd

 <code>Z[3]</code> (abcd
? Нет, в начале 'a', тут 'd'). Совпало 3 бу
 <code>Z[4], Z[5], Z[6]</code>

 </div>
</details>
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

```
</div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
```

```
 <div class="bg-slate-800 p-6 rounded-lg border bo
```

```
 <div class="absolute -top-3 left-4 bg-slate-700
```

```
 emerald-500 shadow-md">
```

```
 Аналогия 1: Паутина (Бор)
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Представ
```

```
 ет — мы падаем по суффиксной ссылке ("
```

```
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2
```

```
 <div class="absolute -top-3 left-4 bg-rose-900
```

```
 -500 shadow-md">
```

```
 Аналогия 2: Матёшка (Терминальные)
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Представ
```

```
 нашли и "he", потому что оно спрятано внутри
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border b
```

```
 <summary class="p-4 font-bold text-slate-400 ou
```

```
 r-slate-700/50">
```

```
 Код (Скрыто)
```

```
 </summary>
```

```
 <div class="p-5 text-sm text-slate-300 space-y-
```

```
 <pre class="bg-slate-950 p-4 rounded overfl
```

```
int get_link(int v) {
```

```
 if (t[v].link == -1) {
```

```
 if (v == 0 || t[v].p == 0) t[v].link = 0;
```

```
 else t[v].link = go(get_link(t[v].p), t[v].pch)
```

```
 }
```

```
 return t[v].link;
```

```
}
```

```

 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP – это
 полную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4
 order border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ь отличный переводчик на английский (Сведения
 аче В, то В – это NP-Полная (сосредоточена
 </div>
 </div>
 </div>
</section>`
 }
};

```

ФАЙЛ 26/83: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">

```

```

<!-- Аналогия DFS -->
<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000080; border-radius: 10px; box-shadow: 5px 5px 0px #000080;">
 ↗ DFS (Поиск в глубину) = Жадный алгоритм
</div>
<p class="text-slate-300 text-sm mb-2">
 Что делает: Жадный алгоритм находит ближайшую свободную вершину (например, минимальную по номеру) и пробует другой путь.
</p>
<ul class="text-xs text-indigo-300 list-disc" style="list-style-type: none;">
 Где используется:
 Топологическая сортировка (DFS)
 Поиск мостов и точек сочленения
 Сильно связанные компоненты Косараса
 Поиск циклов и просто проверка связности

</div>

```

```

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 ↗ BFS (Поиск в ширину) = Волна
</div>
<p class="text-slate-300 text-sm mb-2">
 Что делает: Концентрические волны. Работает через Очередь (FIFO). Продвигается от начальной точки во все стороны.
</p>
<ul class="text-xs text-emerald-300 list-disc" style="list-style-type: none;">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Алгоритм Ахо-Корасик (сборка слов)
 Алгоритм Форда-Фалкерсона / Поиск в ширину
 Двунаправленный поиск для ускорения

</div>

```

```

</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">Эта теорема утверждает, что любой планарный граф можно раскрасить 4 цветами.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">Границы не пересекаются в одной точке по-
 тому что мы не сливались цветами, Всегда можно всего 4
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {

```

```
description: "Разложение задач по порядку выполнения",
category: "Графы",
content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 <div class="bg-slate-900/40 rounded-lg p-6">
 <summary class="p-4 font-bold text-slate-300">

```

```
description: "Разбиение орграфа на макро-вершины. А
category: "Графы. Продвинутые",
content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компоненты
и <i>ориентированного</i> графа, в котором
 </div>
 </div>

 <p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300">
 Если у тебя есть цикл <code>А → Б → А</code>
 Если ты добавишь вершину Г к
 четыре вершины – одна большая SCC.
 Правило: Если два цикла имеют хотя бы

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
 Аналогия: Интриги в офисе
 </div>
 <p class="text-slate-300 text-sm mb-4">
```



```

 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
 этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {

```

```
<div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком (удаляют вершину) — районы будут хрупкостью системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-emerald-500 shadow-md">
```

```
<div class="absolute -top-3 left-4 bg-emerald-500 border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи — это точки со

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-slate-700/50">
```

```
<summary class="p-4 font-bold text-slate-400">
```

Душный мод: Код и корень DFS (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400 font-bold">Особый
```

```
<p>
```

Для стартовой вершины обхода правил (то выше неё прыгать некуда). Поэтому для него у него больше 1 независимого ребенка

```
</p>
```

```
<div class="bg-slate-950 p-4 rounded-lg">
```

```
<pre class="text-xs font-mono text-slate-300">
```

```
void dfs(int v, int p = -1) {
```

```


 </div>
 </div>
 </section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Эйлеров цикл
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">Задача
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Дана неориентированная графа.
 <p class="text-xl font-mono text-white">Пройти по всем ребрам ровно 1 раз.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая
 </div>
 <p class="text-slate-300 text-sm mb-4">Если графа имеет нечетное число вершин, то не существует замкнутого пути, который
 е сходится нечетное число линий, значит, из каждой вершины должно выйти нечетное
 а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);
edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;

```

```

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки сочленения");
setSteps(newSteps);

```



```

 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-1
/>
<text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
>
 {node.label}
</text>

{isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"

```

```

 <div className="w-3 h-3 rounded-full bg-rose-500">
 Текущая
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border-1 border-rose-500">
 Мост
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded $
xt-slate-500`} `>
 >
 {step.desc}
 </div>
 </div>
 })}
 </div>
 </div>
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition-all"
 style={{
 width: `${(currentStepIndex / Math.max(steps.length, 1)) * 100}%`
 }}
 />
 </div>
 </div>
 </div>
</div>

```



```

 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' for i from 1 to V - 1: // |V|' },
 { line: 4, text: ' for (u, v, weight) in E:' }
];

```

```

 log: `Итерация ${iter}: Проверяем ${edge.u} →
 updated ? ` Релаксация успешна! dist[${edge.v}] =
 dist[${edge.u}] + edge.w,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: ` Ранний выход после итерации ${iter}.
 activeCodeLine: 10
 });
 break;
 }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prevDist },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge} образует цикл.
 activeCodeLine: 9,
 });
 break;
 }
 }

 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prevDist },
 log: ` Проверка завершена. Ни одно ребро не образует цикл.
 activeCodeLine: 10,
 });
 }
}

```



```

 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill=
 ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill=
 split(' ')[0]}</text>
 </g>
);
 }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
 })}
 </div>
)}}
</div>
</div>
</div>
</div>

```

```
 </div>
 </div>
);
 }
}
```

---

ФАЙЛ 29/83: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
}
```

```

steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
});

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добав.
 });
 }
 }
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,

```

```

 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-100 p-4">
 <div className="flex flex-col md:flex-row items-center justify-between">
 <div>
 <h3 className="text-2xl font-bold text-white">
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Вн
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded">
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-blue-500"
 title={isPlaying ? 'Пауза' : 'Воспроизведение'}
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-emerald-500
 ed:hover:bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>
);

```

```

 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors durati
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
 }
 }
 </svg>
</div>

```

```

 { /* Код и Очередь */ }
 <div className="flex flex-col gap-4">

```



```

 <span className="text-slate-500 text-x
) : {
 step.queue.map(({qId, i}) => {
 <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

))
 })
</div>
<div className="mt-2 text-sm text-amber-30
ms-center justify-center">
 "{step.logs}"
</div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 30/83: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

```

```

const imageMap: Record<string, { src: string; alt: string;
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',

```

```

import { ChevronLeft, ChevronRight } from "lucide-react"
import type { Chapter } from "../types";

interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled: hover:text-white disabled:
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1

```

```
 </button>
)}
 </nav>
);
};
```

---

ФАЙЛ 32/83: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useEffect, useRef, useState } from 'react';
import { Check, Download, Loader2, Maximize2, MousePointer } from 'lucide-react';
import type { Chapter } from '../types';
import { useTermHints } from '../hooks/useTermHints';
import { TermPopover, type HintAnchor } from './hints/TermHint';
import { getViz } from './vizRegistry';
import { ChapterNav } from './ChapterNav';
import { downloadChapterHtml } from '../utils/exportHtml';
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator }) => {
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```

 [cancelHide]
);

 const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (el) open(el);
 };

 const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (el) scheduleHide();
 };

 const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
 };

 const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (el) open(el);
 };

 const viz = getViz(chapter.id);

 return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 />
 </div>
 </article>
 </>
);

```

Подчёркнутые термины – интерактивные: наведи  
объяснение на пальцах, мини-анимацию и кнопки

</span>

</p>

{viz && {

<section id="chapter-viz" className="mt-8 scroll-mt-8">

<div className="flex items-center justify-between">

<h3 className="flex items-center gap-2 text-indigo-600">

<Sparkles className="w-4 h-4 text-indigo-600">

Демонстрация: {viz.title}

</h3>

<button

onClick={() => onOpenSimulator(chapterIndex)}>

className="flex items-center gap-1.5 text-indigo-600">

Демонстрация: {viz.title}

>

<Maximize2 className="w-3.5 h-3.5" /> </button>

</div>

</div>

{viz.hint && <p className="text-xs text-slate-500">

<viz.Component />

</section>

&& {

<ChapterNav prev={prev} next={next} index={index}>

</article>

<TermPopover

anchor={anchor}

onClose={() => setAnchor(null)}

onOpenSimulator={(id) => {

setAnchor(null);

onOpenSimulator(id);

}}

onCardEnter={cancelHide}

onCardLeave={scheduleHide}

/>

```
];
```

```
interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}
```

```
const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];
```

```
const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];
```

```
const DFS_STEPS: DfsStep[] = [
 {
```

```

 log: "Видим соседа A (уже посещён)! Обратное ребро
 activeCodeLine: [7,8,9]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
 activeCodeLine: [11,13,16,17]
},

```





```

 return <circle cx={n.x} cy={n.y} r={18} f
 >;
 }}())}
 {GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#fff"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#000"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle"
label}</text>
 {vis && (
 <div
 <rect x={n.x-18} y={n.y-28} width={36} height={10}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 {l:{step.low[n.id]||"?"}}</text>
 </div>
)}
 </g>
)}}
 </svg>
 <div className="flex items-center justify-between" >
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10" >
 <div className="bg-indigo-600 h-full transition-all" >
 </div>
 </div>
 </div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}

```

```

 </div>
 </div>

 { /* Log */ }
 <div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 34/83: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

```

```

 { line: 6, text: ' if d > dist[u]: continue'
 { line: 7, text: ' for v, weight in graph.ne
 { line: 8, text: ' if dist[u] + weight <
 { line: 9, text: ' dist[v] = dist[u]
 { line: 10, text: ' pq.push((dist[v]
 { line: 11, text: ' return dist // Все кратчайши
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999
 const visited: Record<string, boolean> = { A: false
 const prev: Record<string, string | null> = { A: nu

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пи
 activeCodeLine: 2,
 });

```

```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;

```

```

 });

 setSteps(simulationSteps);
 }, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полн
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCu

```

```

 <marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b98b" />
 </marker>
 </defs>
 {/* Рёбра */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v);

 const isOptimal = currentStep.previous[edge.id] === 'optimal';

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b98b' : '#1e293b'}
 strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 strokeDasharray={isChecking ? '4,4' : isOptimal ? '1,1' : '0'}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b98b' : '#1e293b'}
 strokeWidth={isChecking ? 2 : isOptimal ? 1 : 0}
 />
 <text

```

```

 }}}
 </svg>
 <div className="w-full bg-slate-950/80 p-4 rounded-2xl" >
 <p className="font-semibold text-amber-400" >
 <p className="min-h-[40px] flex items-center" >
 </div>
 </div>

 { /* Список смежности */ }
 <div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl" >
 <h4 className="text-lg font-bold text-emerald-400" >
 <div className="space-y-2.5 flex-1 overflow-y-hidden" >
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u
 return (
 <div key={u} className={`p-3 rounded-lg ${
 isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
 }`} >
 <div className="flex items-center gap-2" >

 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2" >
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.v
 return (
 <span key={idx} className={`px-2 py-0.5 rounded ${
 isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
 }`} >
 {edge.v}

 →

 {edge.u}

)
 })}
 </div>
 </div>
)
 })}
 </div>
 </h4>
 </div>

```

```

 </td>
 <td className="py-2.5 px-3 font
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text
 {prev || '-'}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg p
 {codeLines.map((item) => {
 const isActive = currentStep.activeCode
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text
 }`}>
 <span className="text-slate-600 sel
 <span className="whitespace-pre fle
 {isActive && <span className="text-
 }
 </div>
)};
 }}}
 </div>
 </div>
</div>
</div>
</div>

```

```

useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в
codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
 }
 }

```



```

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
 }, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
 <div className="bg-slate-900 rounded-2xl border border-slate-800" >
 <div className="flex flex-col lg:flex-row lg:items-center" >
 <div>
 <div className="flex items-center gap-3 mb-2" >
 <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-white" >
 </h3>
 <p className="text-sm text-slate-400" >
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center gap-2 bg-slate-800" >

```

```

 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.length
 className="p-2 text-slate-400 hover:text-
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded
 <h4 className="text-xs font-bold uppercase trac
 <div className="flex flex-wrap items-center gap
 {Array.from({ length: targetN }, (_, i) => i
 const isCached = num in memoState || num <=
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return {
 <div
 key={num}
 className={`flex flex-col items-center p
 isCurrent
 ? 'bg-emerald-950 border-emerald-500
 : isCached
 ? 'bg-slate-900 border-emerald-500/4
 : 'bg-slate-900/50 border-slate-800
 }`}
 >
 <span className="text-xs text-slate-400
 <span className={`text-xl md:text-2xl font
 {isCached ? val : '0'}
```

```

 : 'border-transparent text-slate-400 hover:
 }` }
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
</div>

```

```

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-between" >
 {activeTab === 'table' && (
 <div className="bg-slate-950 p-6 rounded-2xl border" >
 <h5 className="font-bold text-white text-sm" >

 </h5>
 <p className="text-sm text-slate-400" >
 В классической рекурсии для $N=\{targetN\}$ по
 вычисленное значение в таблице, мы момента
 </p>
 <div className="p-4 bg-slate-900 rounded-xl border" >
 Всего шагов симуляции: {steps.length}

 </div>
 </div>
)}

```

```

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border" >
 <div className="bg-slate-900 px-6 py-3 border" >

 </div>
 <pre className="p-6 font-mono text-sm space-pre-wrap" >
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (

```

---

ФАЙЛ 36/83: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
```

```

<h4 className="text-base font-bold text-white">
<button onClick={resetC1} className="flex items
 old border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> C6poc
</button>
</div>

<div className="bg-slate-950 rounded-xl p-4 borde
 -hidden">
<div className="absolute inset-0 bg-[radial-gra
<svg className="w-full h-[260px] z-10 relative"
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!
 const v = C1_NODES.find(n => n.id === e.v)!
 const key = `${Math.min(e.u,e.v)}-${Math.ma
 const used = c1VisitedEdges.includes(key);
 return <line key={i} x1={u.x} y1={u.y} x2={
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
 }}}
 {C1_NODES.map(n => {
 const isLast = c1Path[c1Path.length-1] === n
 const visited = c1Path.includes(n.id);
 return <g key={n.id} className="cursor-poin
 {isLast && <circle cx={n.x} cy={n.y} r={1
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y+4} textAnchor="middl
 bel}</text>
 </g>;
 }}}
</svg>
<div className="absolute top-2 left-2 bg-slate-
 Pe6ëp: {c1VisitedEdges.length}/{C1_EDGES.leng

```

```

 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф по цвету',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового ребра',
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане, добавление ребра
 Это нарушит структуру дерева.',
},
{
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор?',
 options: [
 'Потому что он написан только для положительных весов',
 'Он жадный и считает, что каждый следующий шаг приведет к оптимальному решению',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на счетах',
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты приедешь раньше, чем раньше,
 ля чего нужен алгоритм Беллмана-Форда.',
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки?',
 options: [
 'Он использует меньше памяти благодаря коммунистическим идеалам',
 'Он умеет работать с отрицательными весами без циклов',
 'Он позволяет выполнять шаги слияния колхозов (коммуны)',
 'Он был разработан в Токио и поэтому самый быстрый',
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая коммуна должна
 делить вычисления на GPU или многопроцессорных кластерах',
},
{

```

```

 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };

const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
};

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">Результаты
 <p className="text-slate-400 text-sm mb-6">Ваш результат: {score} из {quizQuestions.length}
 </p>
 <div className="bg-slate-950 border border-slate-800 p-6">
 <p className="text-slate-400 text-sm uppercase">Ваша оценка:
 <p className="text-5xl font-black text-indigo-500">{score}

```

```

 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer

 let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90"
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border-emerald-500"
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-rose-500"
 } else {
 optionStyle = "bg-slate-800/40 border-slate-800"
 }
 }

 return (
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4 rounded-lg"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full " +
 "border-indigo-500 bg-indigo-500 text-white"}
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div>
 {option}
 </div>
 </div>
)
 })}
 </div>
 <div className="flex-1 text-sm md:text-base">

```



---

ФАЙЛ 38/83: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';

interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6'];
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
```

```

 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]), update: true,
 log: `    Улучшение пути [${i}]→[${j}] через [${k}].`,
 activeCodeLine: 7,
 });
 }
 }
 }

 simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), update: true,
 log: `    Все пары отработаны. Алгоритм Флойда завершён.`,
 activeCodeLine: 8,
 });

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
}, [isPlaying, currentStepIndex, steps]);

```

</div>

```
<svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (
 <g key={` ${u} - ${edge.v} `}>
 <line x1={uNode.x} y1={uNode.y} x2=
 markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x) / 2
 isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x) / 2} y=
 '8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
 });
 })}
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#647481
```



```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
type Action = {
 type: 'visit' | 'process' | 'backtrack';
```

```

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начало`,
 ray.from(vis)}]);

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Действие`,
 q, visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Посещение`,
 tack: [...q], visitedNodes: [...Array.from(vis)] });
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `Очистка`,
 om(vis)}]);

 return steps;
 }
}

```

```

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? generateDfsSteps() : generateBfsSteps(), [mode]);

 useEffect(() => {
 setStepIdx(0);
 }, [steps]);
}

```



```

0_10px_rgba(245,158,11,0.2)]]' : ''}
 `}>
 Узел: {item}
 {idx === (mode === 'dfs'
 <span className="ml-1
)}
 </div>
)}
 </div>
 {(!step.queueOrStack || step.queue
 <div className="text-slate-5
)}
</div>
</div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={() => {setAutoPlay
 "p-2 text-slate-400 hover:text-white hover:l
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={() => setAutoPlay(
 justify-center min-w-[80px] ${mode === 'df
 >
 {autoPlay ? <><Pause size={16}
 </button>
 <button onClick={() => {setAutoPlay
 = steps.length - 1} className="p-2 text-sla
 hover:bg-transparent">
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={() => {setAutoPlay
 g-slate-800 rounded-lg ml-2 border-l border
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

```



```

 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

```

```

const addLog = (msg: string) => setLog(prev => [msg,

// --- INSERT: Новый пациент поступает в приемный пок
const insertPatient = useCallback(({name: string, prio
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выпис
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

const preset = PATIENT_PRESETS[Math.floor(Math.rand
const finalName = name || preset.name.split(' ')[0]
const finalEmoji = preset.emoji;
const finalSymptom = preset.symptom;

const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
};

const nextHeap = heapInsert(heap, newPatient).map(x
// Mark specifically this new patient as 'in' in the
const target = nextHeap.find(x => x.priority === pr
const marked = nextHeap.map(x => x.id === (target?.

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью

setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);

```

```

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animStat
 }));

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animStat
 addLog(` Успех! Пациент ${topPatient.name} п
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
 }, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
 setLog([' База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex flex-col lg:flex-row items-s

 {/* Добавить случайного */}
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to
 opacity-40 disabled:cursor-not-allowed text
 ve:scale-95 transition-all cursor-pointer f
 >
 Привезти по Скорой (INSERT)
 </button>

 {/* Добавить кастомного */}
 <form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border borde
 er-emerald-500 transition-colors placeholde
 />
 <div className="flex flex-col items-center mi
 <span className="text-[9px] font-mono text-
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e
 className="w-20 accent-emerald-500 cursor
 />
 </div>
 <button

```

```

<div className="relative w-full h-full min-h-
 <svg className="absolute inset-0 w-full h-f
 {edges}
 </svg>

 {heap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'anim
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400">

```

```

 <div className="text-center text-slate-600">

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Ждём са
 </p>
 </div>
 </div>
 </div>
 </div>

 <div className="w-full h-3 bg-gradient-to-r f
 </div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-emerald-!
 {entry}
 </div>
 })}
 </div>
</div>
);
};

```

ФАЙЛ 41/83: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { createContext, useContext, useEffect, u
```

```

 done: "#10b981",
 warn: "#f59e0b",
 bad: "#f43f5e",
 edge: "#475569",
 };

export const Svg: React.FC<{ children: React.ReactNode;
 const paused = useContext(DemoPauseContext);
 return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза – курсор на карточке" : "наведи"}
 </div>
 </div>
);
};

```

```

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </circle>
 </g>
);

```

```

const viaOpen = step >= 1;
const relaxed = step >= 2;
return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь $i \rightarrow j = 9$ "
 : step === 1
 ? "Разрешаем пересадку через k: $3 + 4 = 7$ "
 : " $7 < 9 \rightarrow D[i][j] = 7$ "
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],

```



```

 : "Вес ребра – цена перехода"
 }
>
 {edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
 {step >= 2 && (
 <text
 x={({a[0] + b[0]) / 2}
 y={({a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
)}
);
 </Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

```

```

)))
 </Svg>
);
};

/* ----- Splay: три случая поворота -----

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, string][];
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#f1c40f";

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

 }}
 />
};

```

ФАЙЛ 43/83: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "A"],
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order[step - 1] : null;
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`}>
 {GRAPH_EDGES.map(([u, v], i) => (
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
))}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k}
 fill={wave?.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
 stroke={wave?.includes(k) ? "#a5b4fc" : undefined} />
))}
 </Svg>
);
};

```

```

 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} fill={C.active} />
 <text x={n.x} y={n.y} textAnchor="middle" fontSize={14}>{n.id}</text>
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={10} fill={C.active} />
 {idx >= 0 && (
 <text x={n.x + 30} y={n.y - 12} textAnchor="middle" fontSize={14}>{n.id}</text>
)}
 </g>
);
 }
 }
 <text x={224} y={62} textAnchor="middle" fontSize={14}>12 > 3 + 4 = 7</text>
 <text x={224} y={74} textAnchor="middle" fontSize={14}>лучше</text>
 </Svg>
);
};

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь улучшился" : ""}>
 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "green" : "red"} />
 <text x={130} y={54} textAnchor="middle" fontSize={14}>12</text>
 <Node x={60} y={70} r={20} fill={C.active} label="12" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="7" />
 <text x={60} y={106} textAnchor="middle" fontSize={14}>3</text>
 <text x={200} y={106} textAnchor="middle" fontSize={14}>4</text>
 <text x={130} y={22} textAnchor="middle" fontSize={14}>лучше</text>
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);

```

```

 })
 </Svg>
);
};

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",

```

```

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",

```

```

 });
};

export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], [
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮЮ тара."
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={
 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
))}
 <text x={130} y={16} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeigh
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХ
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХ
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>

```

```
export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общие

 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 </Svg>
);
};
```

```
export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)})
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};
```

```
export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3
 const step = useLoop(links.length + 1, 900);
 return (
```

```

 </g>
)})
 </Svg>
);
};

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
 : `о префиксные суммы P`} >
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim} />
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "fill 0.2s" }} />
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```



```

const step = useLoop(2, 1400);
return (
 <Svg caption={step === 0 ? "Найти решение – долго ("}
 <ellipse cx={130} cy={64} rx={112} ry={48} fill="white" />
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" />
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle" />
 <text x={206} y={34} textAnchor="middle" fontSize={14} />
 <circle cx={196} cy={84} r={13} fill={step === 1 ? "red" : "white"} />
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle" />
 ?"></text>
 <text x={130} y={124} textAnchor="middle" fontSize={14} />
 </Svg>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс` : ""}
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - len)} ? "red" : "white"}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle" />
 <text x={32 + idx * 33} y={70} textAnchor="middle" />
 fill={idx <= i && step > 0 ? C.warn : C.idle} />
 </g>
))}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={14} />
 π[i] – длина самого длинного «префикс = суффикс»
);
};

```

```
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBa
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
```

```
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";

interface Props {
 vizId: string | null;
 onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-en
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 </div>
```

```

 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
 }

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor, ...props }) => {
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number }>({ top: 0, left: 0 });

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });
 }, [anchor]);

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 }, []);

```

```

<div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-gray-600">
 {entry.demo} && <MiniDemo kind={entry.demo} />
 </p>
 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-gray-600">
 {entry.formal}
 </p>
)}
 {entry.complexity && (
 <div className="text-[11px] font-mono text-gray-600">
 Сложность: {entry.complexity}
 </div>
)}
 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-gray-600 font-medium rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
</div>
</div>,
document.body
);
};

```

```

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-width-2px";
 if (step === 2) return "stroke-pink-500 stroke-width-2px";
 return "stroke-slate-700 stroke-dasharray-4 4 stroke-width-2px";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // отсюда и далее
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

```

```

 if (e.id === 'CA' && step > 6) return true;
 return false;
 };

 const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>

```

```

<div className="flex flex-col md:flex-row gap-4" style={
 <div className="bg-[#0f172a] flex-1 min-h-[300px] p-4">
 <svg width="400" height="300" class="h-full w-full">
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5" orient="vertical"
uto-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 1px;" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f1c40f; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5" orient="vertical"
tart-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f08080; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 5" orient="vertical"
o-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #27ae60; stroke-width: 2px;" />
 </marker>
 </defs>

 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

```



```

? 'fill-[#2
: 'fill-sla
/>
<text x={n.x} y={n.y}
-white">
 {n.label}
</text>
 {pot !== null && {
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
 }}
 </g>
)
 }}}
</svg>
</div>

<div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}

```

```

 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;

```

```

 : [...initialEdges],
 });
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый шаг",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
 u,
);
};

```

```

 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в ${sccId}`,
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
 null,

```

```

const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
 </div>
)

```

```

 </marker>
 </defs>

 { /* Edges */
 {initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u);
 const v = initialNodes.find((n) => n.id === edge.v);

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
 }
 })

 // Adjust slightly for bidirectional link
 const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

```

```

 cy={node.y}
 r="20"
 className={`transition-all duration
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
 }}}
</svg>

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">

```

```

 </div>
 </div>

 {/* List of actions log */}
 <div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
 })}
 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 № {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx === steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
 >
 Далее
 </button>
 </div>
 </div>
 </div>
 </div>
</div>

```



```

 color: string;
}

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

```

```
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветны
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цв
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный ц
 type: 'success'
 }, ...prev]));
},
}
```

```
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
},
```

```

 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. С
 type: 'warning'
 }, ...prev]));
 },
 // Step 20: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: ' Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фи
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

```

```

 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону верш
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в куч
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к верш
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-
 type: 'success'

```

```

 description: 'Ребро В-Е отброшено. Плесень не р
 type: 'error'
 }, ...prev]);
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (Е) тянет
 type: 'info'
 }, ...prev]);
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]);
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {

```

```

setVertices(prev => prev.map(v => v.id === 7 ? {
setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
}, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой
 type: 'info'
 }, ...prev]));
},

```

```

 },
 // Step 2: Concurrently merge into Kolkhozes (Component 1) => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/blue)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' };
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' };
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены радостно',
 type: 'success'
 }, ...prev]);
 },
 // Step 3: Five-Year Plan 2 (Inspecting bridge between components) => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}:
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? { ...e, status: 'included' } : e));
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост',
 description: 'Теперь Красный и Синий колхозы одружились. Красный соединился с Синим с весом 5. Остальные мосты B-E(6) и C-F(9) остаются.',
 type: 'warning'
 }, ...prev]);
 },

```



```

 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 изначально имеют свой собственный цвет.
 Мы хотим раскрасить граф так, чтобы
 никакие две смежные вершины не имели
 одинакового цвета.',
 type: 'info',
 },
]);
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 тельную вершину (Heap) и захватываем соседей!',
 type: 'info',
 },
]);
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 рушек имеет свой собственный цвет.
 Чтобы объединиться в колхоз.',
 type: 'info',
 },
]);
 }
};

```

```

const getColorClass = (color: string, id: number) => {
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 }
}

```

```

 Наблюдай за работой Краскала (Раскраска),
 </p>
</div>

```

```

{/* Algorithm Tabs */}
<div className="flex flex-wrap items-center gap-2">
 <div className="flex items-center bg-slate-400 p-2 rounded">
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +

```

```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-3
 /* Graph Canvas */
 <div className="lg:col-span-2 bg-slate-950 rounded-3xl
 active min-h-[480px] overflow-hidden">

 /* Legend */
 <div className="absolute top-4 left-4 bg-slate-900
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 {activeAlgo === 'kruskal' ? (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 </div>
 </div>
 </div>
) : activeAlgo === 'prim' ? (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-1 bg-sky-600">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 </div>
 </div>
) : (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
```

```

 />
 <text
 x={{u.x + v.x} / 2 + "%"}
 y={{u.y + v.y} / 2 + "%"}
 fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: v
 className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -
 }}
 </div>
)}}
</div>

```

```

 }}
 </div>
</div>
)}}

<div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 }
 >
 <div className="flex items-center gap
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCirc
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
)}}
</div>
</div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items

```

```

 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-indigo-400">
 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 ожении:</p>
 <p className="text-xs text-slate-300 leading-5">
 ди и красим вершины в один цвет, бракуя циклы.
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-indigo-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]

def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

2. Жадный выбор ребер
edges.sort() # O(E log E)

```

```

</p>
<pre className="text-xs text-emerald-400" style="font-family: monospace; font-size: 0.8em; margin: 0; padding: 0 10px;">
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}

</pre>
</div>
</div>
</div>
))

{activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" style="background-color: #f0f0f0; padding: 10px; border-radius: 10px;">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl" style="background-color: #1a202c; color: white; padding: 10px;">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-white">
 Boruvka
 </p>
 <p className="text-slate-400 text-xs font-mono">
 p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl" style="background-color: #1a202c; color: white; padding: 10px;">

```

```
 </div>
)}
 </div>

 </div>
);
};
```

---

ФАЙЛ 51/83: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
```



```
 </div>
)})
 </div>
);
}
```

---

ФАЙЛ 52/83: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);
```

```
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
</div>
)}
</>
);
};
```

---

ФАЙЛ 53/83: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Code } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab }) => {
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
```

```

>
 <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 }`}
>
 <Cpu className="w-4 h-4" /> Тренажёры
</button>
<a
 id="download-pdf-btn"
 href="/export/full_code_all_pages.pdf"
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emeral
 title="Скачать полный PDF сборник всех стра
>
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href="/export/full_code_all_pages.txt"
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодо
>
 <FileText className="w-4 h-4" /> TXT

```

```

 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x
 stroke={xing ? "#f43f5e" : "#4f46e5"}
 }
 }
 const labels = ["A","B","C","D","E"];

```

```

 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143","\u04144"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#f43f5e" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20 z-20">
 $V=6, E=9 \Rightarrow 2V-E=8$ (двудольный)
</div>
</div>
</div>

```

```

<div className="bg-amber-950/40 border border-amber-950/40 p-10">
 ▲ Запомни: K_5 и $K_{3,3}$ — два кита непланарности.
 гомеоморфный K_5 или $K_{3,3}$ — он непланарен. Теорема Кюри-Вейля.
</div>

```

```

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍴', color: 'red',
 голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍴', color: 'blue',
 ишу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green',
 ез сметаны, плиз.', animState: 'idle' },
]));

 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

 // ENQUEUE: Встать в конец очереди (хвост / Tail)
 const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяжёло!');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 }, [isServing, queue.length]);

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);

```

```

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: '🍲', color: 'red', text: 'Я ем суп, чтобы не чувствовать голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '🍲', color: 'blue', text: 'Я ем борщ, чтобы не написать ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', text: 'Я ем котлеты без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
 });
};

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В очереди за супом всё честно. Кто первый встал, тот и ест.
 Новые гости встают строго в конец очереди.
 Здесь нет обхода и блата – только строгий порядок.
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 className="text-indigo-400 text-sm font-medium"
 onClick={enqueue}
 >Встать в очередь
 </div>
 </div>
);

```

```

{ /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
<div className="flex flex-col items-center">
 <div className="relative">
 <div className="absolute -top-10 left-10">
 <div className="w-1.5 h-6 bg-slate-400">
 <div className="w-2 h-4 bg-slate-400">
 </div>
 </div>

 </div>
 <div className="text-center">

 ПОВАР

 </div>
 </div>
</div>

{ /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
<div className="flex flex-col items-center">

 </div>

```

```

{ /* СПИСОК ОЧЕРЕДИ */ }
<div className="flex-1 flex items-end gap
 {queue.length === 0 ? {
 <div className="flex-1 flex flex-col
 </
 <p className="text-xs font-bold fon
 <p className="text-[10px]">Все сыты
 </div>
 } : {
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length

```



```


 })
 </div>
 </div>
);
})
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[10px] font-mono"
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center gap-1
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse">
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-between p-4"
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-full"
 {servedHistory.map((item, idx) => {

```

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}
```

```
export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR', 'FISH', 'SALMON']);
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: TrieNode }>({});
 const [simulationStep, setSimulationStep] = useState<number>(0);
 const [bfsQueue, setBfsQueue] = useState<number[]>([]);
 const [currentNode, setCurrentNode] = useState<number>(0);
 const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и начни симуляцию!');
 const [isTalking, setIsTalking] = useState<boolean>(false);
 const [isBlinking, setIsBlinking] = useState<boolean>(false);

 // Регулярное моргание рыбы
 useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 1000);
 return () => clearInterval(blinkInterval);
 }, []);
```

```

 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
 });

 let currentX = 0;
 const paddingX = 70;
 const paddingY = 80;

 const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
 if (leftX === -1) currentX++;
 }
 };

 layoutDFS(0, 0);

 setNodes(newNodes);
 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(

```

```

 rootChildren.forEach(({childId}) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на графике

 уже суффикса просто нет. Жми «Следующий шаг»!'
);
 };

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полный

 граф. Теперь можно полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.children.length} детей)`;

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] != null) {
 v = updatedNodes[v].fail;
 }
 updatedNodes[rNode.fail] = { children: {} };
 updatedNodes[rNode.fail].children[char] = u;
 logs += `Добавляем ребро ${r} -> ${u} с меткой ${char}`;
 }

 setBfsQueue(newQueue);
 setSimulationStep(3);
 setCurrentNode(rNode);
 setSpeechText(logs);
};

```

```

 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45"
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 40 85"
 fill="#be123c"
 className="origin-[35px_100px] animate-
 />

```

```


 </div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 border-2 border-t-transparent min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -transform rotate(-90deg) border-b-[12px] border-b-transparent border-l-[12px] border-l-transparent border-r-[12px] border-r-transparent border-t-[12px] border-t-transparent" />

 <div className="flex items-center space-x-2 text-white">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading-relaxed">
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-slate-700">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Бор готов к старту!' : 'Бор занят!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-2 py-1 shadow-lg shadow-emerald-900/40 transition" />
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stopBFS}
 className="bg-rose-600 hover:bg-rose-500 text-white px-2 py-1 shadow-lg shadow-rose-900/40 transition" />
 <Stop className="w-4 h-4" /> Остановить
 </button>
)}
 </div>
 </div>
 </div>

```

```

return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 {/* Draw Edges */}
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={(parent.x + node.x) / 2 - 10}
 y={(parent.y + node.y) / 2}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
 })}
 </svg>
);

```

```

let fill = '#1e293b';
let stroke = '#475569';

if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
} else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
} else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
}

return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}

```



```

 }}}
 </div>
 </div>

 <form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
 </form>
 </div>

 {/* Таблица Вершин и суффиксных ссылок */}
 <div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex items-center" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>

```

```

 #{node.id}
 {node.id === 0 && <span className=
</td>
<td className="py-2.5 px-3">
 <span className="bg-slate-800 b
 {node.char}

</td>
<td className="py-2.5 px-3 text-s
 {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
 {node.id === 0 ? {
 <span className="text-slate-5
 } : {
 <span className="bg-indigo-95
 #{node.fail}

 }}
</td>
<td className="py-2.5 px-3">
 {node.id === 0 || node.term_lin
 <span className="text-slate-5
 } : {
 <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Ск
 #{node.term_link}

 }}
</td>
<td className="py-2.5 px-3 text-s
 {Object.keys(node.children).len
 ? 'ø'
 : Object.entries(node.children
 .map(([c, id]) => `${c}→#
 .join(', '))}
</td>
<td className="py-2.5 px-3">

```

```

 result?: number; // partial
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];

 return steps;
}
```

```

 codeLine: 2, treeState: tree, highlightNodes: [v]
 }
 return 0;
}
if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v]
 return tree[v];
}
const m = (l + r) >> 1;
steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 6, treeState: tree, highlightNodes: [v]
const left = go(2 * v, l, m, ql, qr);
const right = go(2 * v + 1, m + 1, r, ql, qr);
const res = left + right;
steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`, codeLine: 8, treeState: tree, highlightNodes: [v],
 rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [res]
return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}] = ${ans}`, codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans
return steps;
}

```

```

// 3. Bottom-Up Query steps (iterative on implicit tree)
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

```

```

 v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l == r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
];

```

```

const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];

```

```

const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
];

```

```

];

```

```

const renderNode = useCallback((nd: VNode): React.ReactElement | null => {
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate-200';
 if (isHigh) cls = `${clr.ring} text-white ring-2 shadow-2xl`;
 else if (isChild) cls = `${clr.childRing} text-amber-500`;
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-center justify-center">
 <div className={`flex flex-col items-center justify-center ${cls}`}>
 {nd.l}
 {nd.l}
 {nd.r}
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt-2">
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full">
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
 </div>
);
}, [step, clr]);

return (
 <div className={`rounded-2xl border overflow-hidden ${clr.ring} ${
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 }`} >
 <div className="px-4 py-2.5 flex items-center justify-center">
 <div className={`text-2xl font-extrabold font-mono ${clr.filled} ${
 ? 'text-emerald-950/50' : 'bg-amber-950/50'
 } border-b-2 border-${clr.ring}`}>

```

```
</pre>
```

```

{ /* Description + result */
<div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-900/60" style={{
 px: 3, py: 2.5, bg: 'slate-900/60', border: '1px solid slate-900/60' }}>
 <span className={`mt-0.5 inline-block w-2 h-2 rounded-sm`} style={{
 mt: 0.5, display: 'inline-block', width: 2, height: 2, border: '1px solid',
 borderStyle: 'solid', color: 'emerald' ? 'text-emerald-300' : 'text-slate-400' }}>
 {step.desc}
 {step.result !== undefined && {
 <span className={`ml-auto shrink-0 font-mono`} style={{
 ml: 'auto', shrink: 0, font: 'mono' }}>
 {step.result}

 }}
 </div>

```

```

{ /* Controls */
<div className="px-3 py-2.5 bg-slate-950 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-950" style={{
 px: 3, py: 2.5, bg: 'slate-950', border: '1px solid slate-950' }}>
 <div className="flex items-center bg-slate-900/60 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-900/60" style={{
 px: 3, py: 2.5, bg: 'slate-900/60', border: '1px solid slate-900/60' }}>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} style={{
 px: 1.5, py: 1, outline: '1px solid', borderStyle: 'solid', color: 'white', title: 'Сброс' }}><RotateCcw className="w-3 h-3" />
 <button onClick={() => { setPlaying(false); setSpeed(700); }} style={{
 px: 1.5, py: 1, outline: '1px solid', borderStyle: 'solid', color: 'white', title: 'Норма' }}><RotateCcw className="w-3 h-3" />
 <button onClick={() => setPlaying(!playing)} style={{
 px: 1.5, py: 1, outline: '1px solid', borderStyle: 'solid', color: 'white', title: 'Пауза' }}><Pause className="w-3 h-3" />
 </div>
 <div style="display: flex; align-items: center; gap: 10px;">
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} style={{
 px: 1.5, py: 1, outline: '1px solid', borderStyle: 'solid', color: 'white', title: 'Сброс' }}><RotateCcw className="w-3 h-3" />
 <button onClick={() => { setPlaying(false); setSpeed(700); }} style={{
 px: 1.5, py: 1, outline: '1px solid', borderStyle: 'solid', color: 'white', title: 'Норма' }}><RotateCcw className="w-3 h-3" />
 <button onClick={() => setPlaying(!playing)} style={{
 px: 1.5, py: 1, outline: '1px solid', borderStyle: 'solid', color: 'white', title: 'Пауза' }}><Pause className="w-3 h-3" />
 </div>
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))} style={{
 text: '9px', font: 'bold', rounded: 'px-1.5 py-1 outline-1px solid', color: 'white', borderStyle: 'solid',
 options: [
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
]
 }>
 </div>

```

```

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr),
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr),

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
 /* ---- TOP BAR: array editor + query range ---- */
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 /* Array input */
 <form onSubmit={handleApply} className="flex flex-col">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">
 Array input
 </label>
 <button type="button" onClick={randomize} className="bg-indigo-500 text-white px-2 py-1">
 Refresh
 </button>
 </div>
 <div className="flex gap-2">
 <input
 type="text"
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-500 text-white px-2 py-1">
 Применить
 </button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (

 {v}

))}

 Total: {totalSum}

 </div>
 </form>
 </div>
 </div>
 </div>
)

```



```

{ /* ---- TABS ---- */
<div className="flex flex-wrap items-center gap-1"
 <button onClick={() => setView('build')} className="border-indigo-500 border-2 p-2"
 -colors ${view === 'build' ? 'border-indigo-500' : 'border-slate-200'}`}>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} className="border-emerald-500 border-2 p-2"
 on-colors ${view === 'queries' ? 'border-emerald-500' : 'border-slate-200'}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} className="border-blue-500 border-2 p-2"
 n-colors ${view === 'theory' ? 'border-blue-500' : 'border-slate-200'}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Справ
 </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

```

```

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху

```



```
export const Sidebar: React.FC<SidebarProps> = ({ selectQuery }) => {
 const [query, setQuery] = useState("");
```

```
 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 const map = new Map<string, typeof chapters>();
 for (const c of filtered) {
 const key = c.category?.trim() || "Прочие темы";
 if (!map.has(key)) map.set(key, []);
 (map.get(key) as typeof chapters).push(c);
 }
 return Array.from(map.entries());
 }, [query]);
```

```
 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:p-4">
 <div className="p-4 pb-3 shrink-0">
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute top-0 right-0">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам..."
 className="w-full bg-slate-800/70 border border-slate-500 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2 text-slate-500"
 aria-label="Очистить поиск"
 />
)}
 </div>
 </div>
 </div>
 </aside>
);
}
```

```

 }` }
 />
 </button>
);
 }}}
</div>
</div>
)})
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-l
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex .
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

---

ФАЙЛ 59/83: `src/components/Simulator.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

```

```

interface Plate {
 id: string;
 name: string;
 icon: string;

```

```

 });
 const [newCustomText, setNewCustomText] = useState('');
 const [newCustomProb, setNewCustomProb] = useState(0);
 const [heapMessage, setHeapMessage] = useState<string>('');

 // Stack handlers
 const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: '🍳' },
 { name: 'Кастрюля от супа', icon: '🍲' },
 { name: 'Чашка от кофе', icon: '☕' },
 { name: 'Миска с остатками салата', icon: '🥗' },
 { name: 'Тарелка от тако', icon: '🌮' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
 };

 const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name}`);
 };

 const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
]);
 };

```

```

 });
 setDequeueMessage(" Очередь восстановлена.");
 };

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
 `);

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
 `);

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный

```

```


 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-400'} `}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Queue
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>

 BFS

 </div>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-400'} `}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Heap
 <div className="text-xs opacity-80">Приоритетная очередь
 </div>
 </div>

 DFS

 </div>
 </button>
 </div>
 </div>
 </div>
</div>

```

```

 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {popMessage && {
 <div className="bg-blue-950/60 border borde
imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стон
 <p className="text-xs mt-1">Добавьте гр
 </div>
 } : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 anim
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map((plate, ind
 const isTop = index === 0;

```



```

<div>
 <h3 className="text-lg font-bold text-white">
 Симуляция очереди за супом

 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>
 </div>
 <div className="flex flex-wrap items-center">
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex item-center justify-center gap-2.5 rounded-xl text-sm font-bold shadow-lg"
 >
 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item-center justify-center gap-2.5 rounded-xl border-indigo-500/40 px-4 py-2.5 rounded"
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white transition-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

```

```

{dequeueMessage && {
 <div className="bg-indigo-950/60 border border-indigo-950/60 padding-2">

```



```

 onChange={e => setNewCustomText(e.target.value)}
 placeholder='например: "Привет, я лучший"'
 className="w-full bg-slate-900 border border-emerald-500 transition-colors"
 />
 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center gap-2 text-sm font-medium text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
 </div>
 </form>

```

```

{heapMessage && (
 <div className="bg-emerald-950/60 border border-emerald-500/40 gap-3 animate-fade-in">

 </div>
)}

```

```

 <div>
 <div className={`font-bold text-white`}
 {item.text}
 </div>
 {isTop && (
 <span className="text-[10px] text-white font-mono"
 -0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Будущее

)}
 </div>
 </div>

 <div className="flex items-center justify-between"
 {/* Custom progress bar */>
 <div className="hidden sm:block"
 <div
 className={`h-full rounded-full`}
 style={{ width: `${percent}%` }}
 ></div>
 </div>
 <span className={`font-mono font-mono`}
 {isTop ? 'text-emerald-400 text-sm' : ''}
 >{percent}%
 </div>
 </div>
);
 }
}
</div>
</div>
</div>
</div>
)}
 <div className="mt-6 text-xs text-slate-500"
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
 </div>
</div>
</div>
)}

```

```

 const q = query.trim().toLowerCase();
 if (!q) return all;
 return all.filter(
 (i) =>
 i.entry.title.toLowerCase().includes(q) ||
 (i.entry.hint ?? "").toLowerCase().includes(q)
 (i.chapterTitle ?? "").toLowerCase().includes(q)
);
 }, [query]);

return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabold">
 <p className="text-sm text-slate-400 leading-relaxed">
 Все интерактивные демонстрации в одном месте.
 здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2"
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
 </div>
 </div>
 </div>
)

```

```
 });
};
```

---

ФАЙЛ 61/83: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево.
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
```

```
 * Порядок поворотов выбираешь сам; правильный ли он пока неясно.
```

```
 * только после того, как узел окажется наверху (или по крайней мере в корень).
```

```
 */
```

```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и просека (zig-zag-zig-zag)
```

```
export const PRESETS: { name: string; hintCase: string; target: number; }[] =
```

```
{
```

```
 name: "Линия влево",
```

```
 hintCase: "Zig-Zig",
```

```
 target: 1,
```

```
export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};
```

```
/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}
```

```
/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
 const path = findPath(root, key);
```

```

const rows = Math.max(1, Math.max(...nodes.map((n) =>
const colW = 46;
const rowH = 62;
return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + co
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
});
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.l
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; co
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() =>
 const path = useMemo(() => new Set(findPath(tree, tar

```



```

 >
 <Dices className="w-3.5 h-3.5" /> Другое дере
 </button>
</div>

```

```

<div className="flex gap-2 mb-3 overflow-x-auto n
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11p
 i === presetIdx ? "bg-indigo-600 text-whi
 }`}
 >
 {p.name}
 </button>
)})}
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420)
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}

```

```

<div className="flex flex-wrap items-center gap-2"
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1
 -white disabled:opacity-40 text-xs font-bol
 >
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-1
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-
 : "bg-slate-800 border-slate-700 text-sla
 }`}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5"
 {showAnswer ? "Скрыть разбор" : "Сдаться / по
 </button>

 <span className="text-[11px] text-slate-500 ml-
 поворотов: {moves.length} • минимум: {minimal

</div>

```

```

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40
 Сейчас splay крутил бы узел{" "}
 {expect
 </div>

```

---

ФАЙЛ 62/83: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Что сделано ради понятности:
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
 * * сразу видно, что дерево остаётся деревом поиска;
 * * под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
 * * это и есть доказательство, что поворот ничего не ломает;
 * * кадр «прицел» ничего не двигает, только подсвечивает;
 * * на кадре «результат» остаются призраки – пунктирные узлы в
 * * местах и стрелки «откуда → куда»;
 * * по умолчанию анимация НЕ крутится сама: пользователь
 * * идёт в своём темпе. Автопрокрутка включается кнопкой.
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```

 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду",
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 – 20",
 text: "Ничего пока не двигается. Смотри на подсве",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддерев",
 ева от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {

```

```

 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",
 text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p — дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым — нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,

```

```

 ms: RESULT_MS,
 },
],
 };

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

const COLOR: Record<string, { fill: string; stroke: string }> = {
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id === "B";

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C }> = ({
 frame,
 prev,
 keys,
}) => {
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
 const ghosts = frame.kind === "result" ? (moved ?? []) : [];

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-full h-full">
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" refX="0" refY="0">
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 {/* призраки: где узел стоял до поворота */}
 {ghosts.map((id) => {
 const from = prev[id]!;

```

```

 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 }
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id));
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font-size={10}
 >{id}</text>
)}
 </g>
);
}

```

```

 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 i === caseIdx ? "bg-indigo-600 text-white" : "bg-white text-indigo-600"
 }`}
 >
 {c.label}
 </button>
)})
 </div>

```

```

<div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корпус
 ({active.rotations})
</div>

```

```

{/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
<div className="mb-3 rounded-xl border border-slate-200 p-4">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`}
 >
 {current.kind === "aim" ? "прицел" : current.kind}

 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading-relaxed">
 {current.description}
 </p>
</div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <Tree frame={current} prev={prevPos} keys={activeKeys}

```



```

 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800
 d: hover:text-slate-300 text-xs font-bold tr
>
 <- Назад
</button>
<button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
>
 {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
>
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
</button>
<button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-1
 slow
 ? "bg-emerald-600/20 border-emerald-500 tr
 : "bg-slate-800 border-slate-700 text-sla
 }`}
>
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1
</button>
<button

```

```
 BookOpen,
 } from "lucide-react";

interface SplayNode {
 key: number;
 left: SplayNode | null;
 right: SplayNode | null;
 x?: number;
 y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
 if (!root) return { key, left: null, right: null };
 if (key < root.key) {
 root.left = insertBST(root.left, key);
 } else if (key > root.key) {
 root.right = insertBST(root.right, key);
 }
 return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
 y.right = x;
 return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};
```

```

const steps: string[] = [];
if (!root) return { newRoot: null, steps: ["Дерево"]

// We implement the classic top-down or bottom-up s
// For visualization logs, we do a bottom-up explan
let path: { node: SplayNode; dir: "left" | "right"
let current: SplayNode | null = root;

// Find the element or the element where search fail
let lastNode: SplayNode = root;
while (current) {
 lastNode = current;
 if (key === current.key) {
 path.push({ node: current, dir: null });
 break;
 } else if (key < current.key) {
 path.push({ node: current, dir: "left" });
 current = current.left;
 } else {
 path.push({ node: current, dir: "right" });
 current = current.right;
 }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
 steps.push(`Поиск ${key}: элемент отсутствует в д
 steps.push(
 `Поиск споткнулся на узле [${lastNode.key}]. И
);
} else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем
);
}

```

```

 node.right = rightRotate(node.right);
 steps.push(`Компонент Zig-Zag: правый поворот`);
 }
} else if (k > node.right.key) {
 // Zag-Zag (Right-Right)
 node.right.right = splayAction(node.right.right, target);
 node = leftRotate(node);
 steps.push(
 `Вращение Zag-Zag (Левый поворот родителя для правого ребенка)`
);
}

if (!node.right) return node;
steps.push(
 `Финальный поворот Zag (Левое вращение корня)`
);
return leftRotate(node);
}
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

```

```

 computeCoordinates(node.left, x - levelWidth, y +
 }
 if (node.right) {
 computeCoordinates(node.right, x + levelWidth, y +
 }
};

```

```

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

```

```

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.R
 if (!node) return [];
 let lines: React.ReactNode[] = [];
 if (node.left && node.left.x && node.left.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.left.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.left.x}
 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.left));
 }
 if (node.right && node.right.x && node.right.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.right.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
};

```

```

 {node.key}
 </text>
 </g>,
);
 circles = circles.concat(renderNodes(node.left));
 circles = circles.concat(renderNodes(node.right));
 return circles;
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 { /* Header */ }
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <BookOpen className="w-5 h-5 text-indigo-400"
 Интерактивное Splay-дерево
 </h3>
 <p className="text-xs text-slate-400">
 Кликайте на вершины дерева, чтобы «вытащить»
 Splay.
 </p>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12
 { /* Playfield */ }
 <div className="lg:col-span-7 bg-[#0f172a] p-4
 <svg
 className="w-full h-full max-w-[600px] max-h-[320px]
 viewBox="0 0 600 320"
 >
 {renderLines(drawTree)}
 {renderNodes(drawTree)}
 </svg>

 { /* Quick interactive controls */ }
 <div className="absolute bottom-4 left-4 right-4
 <form onSubmit={handleFind} className="flex
 <input

```

```

 className="flex items-center gap-1.5 px-3
 >
 <RotateCcw className="w-3.5 h-3.5" /> C6p
 </button>
 </div>
 </div>

 { /* Logs */ }
 <div className="lg:col-span-5 bg-slate-950 border
 to overflow-y-auto custom-scrollbar">
 <h4 className="text-xs font-bold text-slate-400">
 Ход вращения (Splay-логи)
 </h4>
 <div className="flex-1 space-y-2 mb-4 overflow
 {log.map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2.5 rounded trans
 idx === log.length - 1
 ? "border-l-2 border-indigo-500 bg-
 : "text-slate-400 bg-slate-900/50"
 }` }
 >
 {step}
 </div>
)} }
 </div>

 <div className="border-t border-slate-800 pt-2">
 <p>

 у самого корня. Делаем 1 вращение.
 </p>
 <p>

 оба левые или оба правые. Вращаем деда, по
 </p>
 <p>

```

</div>

<div className="bg-slate-900/60 p-4 rounded-l

<div>

<p className="font-bold text-slate-300 mb

2. Эффект качелей (Swing)

</p>

<p className="text-slate-400 leading-rela

Если агент запрашивает по очереди{" "

<code className="text-rose-400">[10]</code>

<code className="text-rose-400">[60]</code>

углы), дерево будет превращаться в палку

сторону. Первая операция Splay(60) стои

<strong className="text-white">O(n)</st

глубину n. Вторая Splay(10) заставляет

Постоянный свинг убивает производительн

сложности <strong className="text-white

</p>

</div>

<div className="mt-3 text-[10px] text-rose-

Кэшировать свингующие пары на концах – фа

</div>

</div>

<div className="bg-slate-900/60 p-4 rounded-l

<div>

<p className="font-bold text-slate-300 mb

3. Амортизация  $O(\log n)$

</p>

<p className="text-slate-400 leading-rela

Парадокс: один запрос за{" "

<code className="text-emerald-400">O(n)</code>

всю структуру вдвое более плоской за сч

при двойных вращениях {

<code className="text-emerald-400 font-

). Дорогая операция инвестирует в дешев

операций. Поэтому амортизированное врем

<strong className="text-white"> $O(\log n)$ </strong>



```

 left: SN | null;
 right: SN | null;
 parent: SN | null;
}

export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}

export function bstInsert(root: SN | null, key: number): SN | null {
 if (!root) return { key, left: null, right: null, parent: null };
 const r = cloneWithParents(root)!;
 let cur: SN = r;
 for (;;) {
 if (key < cur.key) {
 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.left;
 } else {
 if (!cur.right) {
 cur.right = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.right;
 }
 }
}

```

```

}

const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
 }
 return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "rot" |
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 pathKeys?: number[];
 caseLabel?: string;
 note: string;
 rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → пары (п
export function splaySteps(
 rootIn: SN | null,
 key: number,
 isInsert = false,
): { steps: SplayStep[]; found: boolean } {
 let root: SN | null = cloneWithParents(rootIn);
 const steps: SplayStep[] = [];
 if (!root) return { steps: [{ kind: "done", root, note: "" }], found: false };

 // --- спуск ---
 let cur: SN | null = root;
 const pathKeys: number[] = [];

```

```

 caseLabel: "Zig",
 note: `Zig: у ${target.key} нет деда — хватит о,
 rotCount,
 });
 const before = collectEdges(root);
 rotateUp(target);
 fixRoot();
 rotCount += 1;
 steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: "Zig",
 note: `Поворот: ${diffEdges(before, collectEdge
 rotCount,
 });
 continue;
}
const pIsLeft = g.left === p;
const xIsLeft = p.left === target;
if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [g.key, p.key],
 caseLabel: "Zig-Zig",
 note: `Zig-Zig (${side}, линия ${g.key}—${p.key
 а ${target.key} пока не трогаем.`,
 rotCount,
 });
 let before = collectEdges(root);
 rotateUp(p);
 fixRoot();
 rotCount += 1;
 steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: "Zig-Zig",

```

```

 root: cloneWithParents(root)!,
 caseLabel: "Zig-Zag",
 note: `Поворот 1: ${diffEdges(before, collectEdges(root))}`,
 rotCount,
));
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Вроде ребро: ${target.parent.key}—${target.key}`,
 rotCount,
 });
 before = collectEdges(root);
 rotateUp(target);
 fixRoot();
 rotCount += 1;
 steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: "Zig-Zag",
 note: `Поворот 2: ${diffEdges(before, collectEdges(root))}`,
 rotCount,
));
 }
}

steps.push({
 kind: "done",
 root: cloneWithParents(root)!,
 note: ` ${target.key} в корне за ${rotCount} ${rotCount} шагов`,
 rotCount,
});
return { steps, found };
}

/* ----- служебное -----

```

```

const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * SLOT_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>();
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>();

 const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key, isInsert);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
 };

 useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 // фиксируем результат в базовом дереве
 }
 }, [playing, steps, idx]);
};

```

```

const withNew = bstInsert(baseRef.current, v);
baseRef.current = withNew;
setTreeVers((t) => t + 1);
runKey(v, true);
setInput("");
};

const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
 if (step.aimEdge && step.aimEdge.includes(k)) return "#0f172a";
 return "#0f172a";
};

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind !== "search" && step.pathKeys?.includes(k)) return "#64748b";
 if (step.aimEdge?.includes(k)) return "#f59e0b";
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Шанка */}
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Клик
 </div>
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3" >
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 </div>
 </div>
 </div>
);

```

```

 { /* Плеер */
 {steps && (
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setIdx(0); setPlaying(true); }}>
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }}>
 Пауза
 </button>
 <button onClick={() => setPlaying(!playing)}>
 {playing ? "▶ Пуск" : "⏸ Пауза"}
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }}>
 Стоп
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && (

 {step.caseLabel}

)}
 </div>
)}
)}

```

```

 { /* Дерево */
 <div className="bg-indigo-950/20 rounded-xl p-4 border">
 {!steps && (
 <div className="absolute top-3 left-3 bg-indigo-950/20">
 Кликни по любому узлу – splay
 </div>
)}
 </div>
)}

```

```

 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}}}
</svg>

{/* Лог шага */}
<div className="w-full bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400 mb-2"
 {step?.kind === "aim" ? " Прицел:" : step?.kind}
 >
 </p>
 <p className="min-h-[40px] flex items-center"
 ельности: прицел на ребро, поворот, переехал
 </p>
</div>
</div>

{/* Счётчики + код */}
<div className="flex flex-col lg:flex-row gap-6"
 <div className="w-full lg:w-1/2 bg-rose-950/10"
 <h4 className="text-sm font-bold text-rose-300"
 {steps ? (
 <table className="w-full text-sm"
 <tbody className="font-mono"
 <tr className="border-b border-slate-800"
 <td className="py-2 text-slate-400">р.
 <td className="py-2 text-right text-slate-400">
 </td>
 </tr>
 <tr className="border-b border-slate-800"
 <td className="py-2 text-slate-400">в.
 <td className="py-2 text-right text-slate-400">

```



```
export default SplayWalkViz;
```

---

ФАЙЛ 65/83: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
```

```
interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}
```

```
const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pasta' },
 { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pizza' },
 { name: 'Блюдец от торта', emoji: '🍰', color: 'from-pie' },
 { name: 'Тарелка от тако', emoji: '🌮', color: 'from-yaki' },
 { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-sushi' },
 { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-pan-fry' },
];
```

```
let counter = 0;
```

```
export const StackViz: React.FC = () => {
 const [dirtyStack, setDirtyStack] = useState<Plate[]>([
 { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'idle' },
])
```

```

 }, [dirtyStack, isWashing]));

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

 // Step 1: Start washing animation (sponge and soap)
 setDirtyStack(prev => prev.map(p => p.id !== topPlate.id));
 setShowSoap(true);

 setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',
 };

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, 10));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
 }, 850);
}, [dirtyStack, isWashing]);

const reset = () => {

```

```

 acy-40 disabled:cursor-not-allowed text-w
 ale-95 transition-all cursor-pointer flex i
 >
 Положить грязную (PUSH)
</button>

<button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-900
 opacity-40 disabled:cursor-not-allowed text-white
 ve:scale-95 transition-all cursor-pointer flex items-center justify-center"
 >
 Помыть верхнюю (POP)
</button>

<button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white
 cursor-pointer border border-slate-700"
 >
 Сброс
</button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12 gap-4">

 {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
 <div className="md:col-span-7 bg-slate-900 rounded-2xl p-4
 [380px]">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 <div className="absolute top-4 right-4 text-[10px] font-mono">
 Визуализация раковины
 </div>
 </div>
 <div className="md:col-span-5 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Грязная стопка (Стек вызовов / LIFO)
 </div>
 <div className="absolute top-4 right-4 text-[10px] font-mono">
 Визуализация раковины
 </div>
 </div>
</div>

```



```

 <div className="text-[10px] font-mono text-slate-500">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`}
 >
 {idx === 0 ?
 {entry}
 </div>
)}}
 </div>
 </div>
);
};

```

ФАЙЛ 66/83: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const pi = new Array(n).fill(0);
 const steps: any[] = [];

 steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

 for (let i = 1; i < n; i++) {
 let j = pi[i - 1];

 while (s[i] !== s[j]) j = pi[j - 1];

 steps.push({ i, j, pi: [...pi], highlight: [i, j] });
 }
}

```

```

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Предел достигнут. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i });
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i], desc: `Окно падает! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i], desc: `Окно падает! Окно растёт.` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упёрся в стену.` });
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксируем текущее состояние.` });

 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 steps.push({ i, l, r, z: [...z], desc: `Обновляем границы.` });
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true });
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

const nextStep = () => { setAutoPlay(false); setStepId
const prevStep = () => { setAutoPlay(false); setStepId
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <div className="flex bg-slate-900 border border-slate-800 rounded-xl px-3 py-1.5 text-white text-xs font-bold font-mono px-2 py-1 outline">
 <button onClick={() => setMode("kmp")}
 className={`px-3 py-1.5 text-xs font-mono`}
 e" : "text-slate-400 hover:text-slate-300">
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")}
 className={`px-3 py-1.5 text-xs font-mono`}
 " : "text-slate-400 hover:text-slate-300">
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2">
 <div className="bg-slate-800 p-1 flex items-center gap-1">

 <input value={pattern} onChange={setPattern} type="text" />
 </div>
 <div className="bg-slate-800 p-1 flex items-center gap-1">

 <input value={text} onChange={setText} type="text" />
 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl border border-slate-800 px-3 py-1.5 flex items-center justify-start">

```

```

 <div className="abs
 }}

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

 {/* Value array box */}
 <div className="text-[1
slate-700/50 font-mono font-bold">
 {mode === "kmp" ? s
 </div>

 {/* KMP fingers */}
 {mode === "kmp" && step
 <div className="abs

z-20">
 <span classNam
 </div>
 }}
 {mode === "kmp" && step
 <div className="abs
 <span classNam
 </div>
 }}
 {mode === "kmp" && step
 <div className="abs

20">
 <span classNam
 </div>
 }}

 {/* Z fingers */}

```



```

 </button>
 <button onClick={playPause} className=
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16}
 </button>
 <button onClick={nextStep} disabled:
r:bg-slate-800 rounded-lg disabled:opacity-1
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={reset} className="
border-slate-800">
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50
erflow-hidden">
 <div className="absolute top-0 righ
 <span className="font-mono text
 </div>
 <div className="text-[10px] text-sla
{steps.length}]</div>
 <div className="text-sm text-slate-1
 {step.desc}
 </div>
 </div>
 </div>

 {/* Why this matters card */}
 <div className="bg-slate-800 p-4 rounded-xl

 <p className="text-xs text-slate-400 le
 Обрати внимание на хитрость с решёт
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "пр
 о было найдено!
 Пройди шаги вручную, посмотри, как

```

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}

const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];

interface TStep {
```

```

dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in DAG)
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
 u,
);
 }
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in reverse
recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'`,
 u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой вершины ${i}`,
 null,
);
 dfs(i);
 }
}

```

```

 };

 const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>

```

```

 (step.currentNode === edge.u || step.currentNode === edge.v ||
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow-visited"})`}
 className="transition-all duration-300"
 />
);
 });
}

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited = step.visited.has(node.id);
 const isProcessed = step.fullyProcessed.has(node.id);

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";

 void textColor;

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#f8d7da";
 stroke = "#f5c6cb";
 textColor = "text-red-500 font-italic";
 }

 return (
 <div
 key={node.id}
 style={{
 position: "relative",
 width: 40px,
 height: 40px,
 margin: 10px,
 display: "flex",
 align-items: "center",
 justify-content: "center",
 }}
 >
 <div
 style={{
 position: "absolute",
 top: -10px,
 left: -10px,
 width: 20px,
 height: 20px,
 background-color: "white",
 border: 1px solid "black",
 border-radius: "50%",
 display: "flex",
 align-items: "center",
 justify-content: "center",
 font-size: 10px,
 font-weight: "bold",
 color: "black",
 }}
 />
 <div
 style={{
 position: "absolute",
 top: -10px,
 right: -10px,
 width: 20px,
 height: 20px,
 background-color: "white",
 border: 1px solid "black",
 border-radius: "50%",
 display: "flex",
 align-items: "center",
 justify-content: "center",
 font-size: 10px,
 font-weight: "bold",
 color: "black",
 }}
 />
 <div
 style={{
 position: "absolute",
 bottom: -10px,
 left: -10px,
 width: 20px,
 height: 20px,
 background-color: "white",
 border: 1px solid "black",
 border-radius: "50%",
 display: "flex",
 align-items: "center",
 justify-content: "center",
 font-size: 10px,
 font-weight: "bold",
 color: "black",
 }}
 />
 <div
 style={{
 position: "absolute",
 bottom: -10px,
 right: -10px,
 width: 20px,
 height: 20px,
 background-color: "white",
 border: 1px solid "black",
 border-radius: "50%",
 display: "flex",
 align-items: "center",
 justify-content: "center",
 font-size: 10px,
 font-weight: "bold",
 color: "black",
 }}
 />
 <div
 style={{
 position: "absolute",
 top: 50%,
 left: 50%,
 transform: "translate(-50%, -50%)",
 font-size: 12px,
 font-weight: "bold",
 color: textColor,
 }}
 />
 </div>
);
})}

```

```

 </text>
 </g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800">
 <p className="text-xs text-indigo-300 min-h-12">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>
 </div>

```

```
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 68/83: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useMemo, useState } from "react";
```

```
/**
```

```
 * Интерактивный Треп – шесть режимов:
```

```
 * build – сборка: ВИДИМАЯ сортировка точек (и чем с
```

```
 * координатная плоскость НАВЕРХУ (ассоциаци
```

```
 * чек-лист инвариантов (автопроверка, как у
```

```
 * split – разрез по x с двумя корзинами (грамотная
```

```
 * merge – склейка двух деревьев по y .
```

```
 * erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x ,
```

```
 * layers – миф «дети в массиве по $2k/2k+1$ »: работает
```

```
 * search – «отсортирую бинарным поиском»: поиск не с
```

```
 * сортировка платит $\geq n \cdot \log n$ сравнений, со
```

```
 * сортировка платит $\geq n \cdot \log n$ сравнений, со
```

```
 * Канонический набор: все x и все y различны $\Rightarrow$ дерево
```

```
 * Узел идентифицируется парой $id = "x:y"$.
```

```
 */
```

```
export interface Pair {
 key: number;
 grade: number;
```

```

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort(

/* ----- чистые алгоритмы (экспортируются дл

export function insert(root: TNode | null, p: Pair): TNode {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
}

export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null;
 for (const p of sortedByY(ps)) root = insert(root, p);
 return root;
}

export interface SplitResult {
 l: TNode | null;
 r: TNode | null;
 trace: { atId: NodeId; goLeft: boolean; note: string }[];
}

export function splitTree(root: TNode | null, x: number): SplitResult {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
 if (!t) return { l: null, r: null };
 if (t.key <= x) {
 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key} > x)
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key} < x)
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 }
}
```



```

 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли (${t.key}), ${t.key} <=${key}` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
 return m.root;
};
return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
 t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

/\* ----- валидация treap (для чек-листа и стресс-теста) ----- \*/

```

export function checkTreap(root: TNode | null): { bst: boolean, heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null): void => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

```

const TreeView: React.FC<TreeViewProps> = ({ root, width, height }) => {
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([id, n]) => n));
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px + n.rx));
 const autoH = Math.max(260, ...nodes.map((n) => n.py + n.rh));
 const w = width ?? autoW;
 const h = height ?? autoH;
 const byId = (id: NodeId) => nodes.find((n) => n.id === id);
 return (
 <div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
 {edges.map((e) => {
 const a = byId(e.from);
 const b = byId(e.to);
 if (!a || !b) return null;
 return (
 <line
 key={`te-${e.from}-${e.to}`}
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={edgeColor ? edgeColor(e.from, e.to) : 'black'}
 strokeWidth={2.5}
 strokeDasharray={edgeDash ?? edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
);
 })}
 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`}
 <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
 />
 <rect x={-10} y={-10} x2={10} y2={10} fill="white" stroke="black"
 />
 <text y={-2} fill="#fff" fontSize="13" font-weight="bold">{n.name}</text>
 <text y={13} fill="#cbd5e1" fontSize="10">{n.id}</text>
);
 })}
 </svg>
 </div>
);
};

```

```

 className={`w-full ${interactive ? "touch-none select-none" : ""}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" strokeDasharray="4 3"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" strokeDasharray="4 3"/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" strokeDasharray="4 3"/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(0)} stroke="black" strokeDasharray="4 3"/>
 {[0, 10, 20, 30, 40].map((k) => {
 <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} fill="black" fontFamily="monospace" fontStyle="italic" fontWeight="bold" fontSize="12"/>
 })}
 {[-4, 0, 4, 8].map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} fill="black" fontFamily="monospace" fontStyle="italic" fontWeight="bold" fontSize="12"/>
 })}
 <text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8" fontFamily="monospace" fontStyle="italic" fontWeight="bold" fontSize="12">P</text>
 <text x={PX(1) - 26} y="14" fill="#94a3b8" fontFamily="monospace" fontStyle="italic" fontWeight="bold" fontSize="12">1</text>

 {drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.a.grade === e.b.grade) || (badEdge.a.key === e.b.key && badEdge.a.grade === e.a.grade));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} stroke="black" strokeDasharray={isBad ? "4,3" : "none"} strokeWidth={2}/>
 })}

 {/* временная линия игры */}
 {interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDasharray="4 3"
 />
)}
 </div>
)

```

```

const es: GEdge[] = [];
const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.g
 if (n.right) { es.push({ a: { key: n.key, grade: n.g
};
walk(buildTree(POINTS));
return es;
})();

const sameP = (p: Pair, q: Pair) => p.key === q.key &&
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
 edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa
 let cur: Pair | undefined = from;
 const seen = new Set<string>();
 while (cur) {
 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur))?.a;
 }
 return false;
};

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 спр
export function forestIsBst(edges: GEdge[], all: Pair[])
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.
 const hasParent = new Set(edges.map((e) => gpid(e.b))
 const roots = all.filter((p) => !hasParent.has(gpid(p
 const ok = (n: Pair | undefined, lo: number, hi: numb
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n)) ?? []).slice().sort((

```

```

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
 for (const chunk of chunks) {
 const nums = chunk.split(/\sxx*/).filter(Boolean);
 if (nums.length !== 2 || nums.some((v) => !Number.isInteger(+v))) {
 errors.push(`«${chunk}» – нужно ровно два числа: два числа и пробел`);
 continue;
 }
 const id = `${nums[0]}:${nums[1]}`;
 if (seen.has(id)) {
 errors.push(`точка (${nums[0]}; ${nums[1]}) дважды`);
 continue;
 }
 seen.add(id);
 pairs.push({ key: nums[0], grade: nums[1] });
 }
 const xs = pairs.map((p) => p.key);
 const ys = pairs.map((p) => p.grade);
 return { pairs, errors, equalX: new Set(xs).size !== 1, equalY: new Set(ys).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0, 1 1, 1 2, 1 3, 1 4, 1 5, 1 6, 1 7, 1 8, 1 9, 1 10, 1 11, 1 12, 1 13, 1 14, 1 15, 1 16, 1 17, 1 18, 1 19, 1 20, 1 21, 1 22, 1 23, 1 24, 1 25, 1 26, 1 27, 1 28, 1 29, 1 30, 1 31, 1 32, 1 33, 1 34, 1 35, 1 36, 1 37, 1 38, 1 39, 1 40, 1 41, 1 42, 1 43, 1 44, 1 45, 1 46, 1 47, 1 48, 1 49, 1 50, 1 51, 1 52, 1 53, 1 54, 1 55, 1 56, 1 57, 1 58, 1 59, 1 60, 1 61, 1 62, 1 63, 1 64, 1 65, 1 66, 1 67, 1 68, 1 69, 1 70, 1 71, 1 72, 1 73, 1 74, 1 75, 1 76, 1 77, 1 78, 1 79, 1 80, 1 81, 1 82, 1 83, 1 84, 1 85, 1 86, 1 87, 1 88, 1 89, 1 90, 1 91, 1 92, 1 93, 1 94, 1 95, 1 96, 1 97, 1 98, 1 99, 1 100, 1 101, 1 102, 1 103, 1 104, 1 105, 1 106, 1 107, 1 108, 1 109, 1 110, 1 111, 1 112, 1 113, 1 114, 1 115, 1 116, 1 117, 1 118, 1 119, 1 120, 1 121, 1 122, 1 123, 1 124, 1 125, 1 126, 1 127, 1 128, 1 129, 1 130, 1 131, 1 132, 1 133, 1 134, 1 135, 1 136, 1 137, 1 138, 1 139, 1 140, 1 141, 1 142, 1 143, 1 144, 1 145, 1 146, 1 147, 1 148, 1 149, 1 150, 1 151, 1 152, 1 153, 1 154, 1 155, 1 156, 1 157, 1 158, 1 159, 1 160, 1 161, 1 162, 1 163, 1 164, 1 165, 1 166, 1 167, 1 168, 1 169, 1 170, 1 171, 1 172, 1 173, 1 174, 1 175, 1 176, 1 177, 1 178, 1 179, 1 180, 1 181, 1 182, 1 183, 1 184, 1 185, 1 186, 1 187, 1 188, 1 189, 1 190, 1 191, 1 192, 1 193, 1 194, 1 195, 1 196, 1 197, 1 198, 1 199, 1 200, 1 201, 1 202, 1 203, 1 204, 1 205, 1 206, 1 207, 1 208, 1 209, 1 210, 1 211, 1 212, 1 213, 1 214, 1 215, 1 216, 1 217, 1 218, 1 219, 1 220, 1 221, 1 222, 1 223, 1 224, 1 225, 1 226, 1 227, 1 228, 1 229, 1 230, 1 231, 1 232, 1 233, 1 234, 1 235, 1 236, 1 237, 1 238, 1 239, 1 240, 1 241, 1 242, 1 243, 1 244, 1 245, 1 246, 1 247, 1 248, 1 249, 1 250, 1 251, 1 252, 1 253, 1 254, 1 255, 1 256, 1 257, 1 258, 1 259, 1 260, 1 261, 1 262, 1 263, 1 264, 1 265, 1 266, 1 267, 1 268, 1 269, 1 270, 1 271, 1 272, 1 273, 1 274, 1 275, 1 276, 1 277, 1 278, 1 279, 1 280, 1 281, 1 282, 1 283, 1 284, 1 285, 1 286, 1 287, 1 288, 1 289, 1 290, 1 291, 1 292, 1 293, 1 294, 1 295, 1 296, 1 297, 1 298, 1 299, 1 300, 1 301, 1 302, 1 303, 1 304, 1 305, 1 306, 1 307, 1 308, 1 309, 1 310, 1 311, 1 312, 1 313, 1 314, 1 315, 1 316, 1 317, 1 318, 1 319, 1 320, 1 321, 1 322, 1 323, 1 324, 1 325, 1 326, 1 327, 1 328, 1 329, 1 330, 1 331, 1 332, 1 333, 1 334, 1 335, 1 336, 1 337, 1 338, 1 339, 1 340, 1 341, 1 342, 1 343, 1 344, 1 345, 1 346, 1 347, 1 348, 1 349, 1 350, 1 351, 1 352, 1 353, 1 354, 1 355, 1 356, 1 357, 1 358, 1 359, 1 360, 1 361, 1 362, 1 363, 1 364, 1 365, 1 366, 1 367, 1 368, 1 369, 1 370, 1 371, 1 372, 1 373, 1 374, 1 375, 1 376, 1 377, 1 378, 1 379, 1 380, 1 381, 1 382, 1 383, 1 384, 1 385, 1 386, 1 387, 1 388, 1 389, 1 390, 1 391, 1 392, 1 393, 1 394, 1 395, 1 396, 1 397, 1 398, 1 399, 1 400, 1 401, 1 402, 1 403, 1 404, 1 405, 1 406, 1 407, 1 408, 1 409, 1 410, 1 411, 1 412, 1 413, 1 414, 1 415, 1 416, 1 417, 1 418, 1 419, 1 420, 1 421, 1 422, 1 423, 1 424, 1 425, 1 426, 1 427, 1 428, 1 429, 1 430, 1 431, 1 432, 1 433, 1 434, 1 435, 1 436, 1 437, 1 438, 1 439, 1 440, 1 441, 1 442, 1 443, 1 444, 1 445, 1 446, 1 447, 1 448, 1 449, 1 450, 1 451, 1 452, 1 453, 1 454, 1 455, 1 456, 1 457, 1 458, 1 459, 1 460, 1 461, 1 462, 1 463, 1 464, 1 465, 1 466, 1 467, 1 468, 1 469, 1 470, 1 471, 1 472, 1 473, 1 474, 1 475, 1 476, 1 477, 1 478, 1 479, 1 480, 1 481, 1 482, 1 483, 1 484, 1 485, 1 486, 1 487, 1 488, 1 489, 1 490, 1 491, 1 492, 1 493, 1 494, 1 495, 1 496, 1 497, 1 498, 1 499, 1 500, 1 501, 1 502, 1 503, 1 504, 1 505, 1 506, 1 507, 1 508, 1 509, 1 510, 1 511, 1 512, 1 513, 1 514, 1 515, 1 516, 1 517, 1 518, 1 519, 1 520, 1 521, 1 522, 1 523, 1 524, 1 525, 1 526, 1 527, 1 528, 1 529, 1 530, 1 531, 1 532, 1 533, 1 534, 1 535, 1 536, 1 537, 1 538, 1 539, 1 540, 1 541, 1 542, 1 543, 1 544, 1 545, 1 546, 1 547, 1 548, 1 549, 1 550, 1 551, 1 552, 1 553, 1 554, 1 555, 1 556, 1 557, 1 558, 1 559, 1 560, 1 561, 1 562, 1 563, 1 564, 1 565, 1 566, 1 567, 1 568, 1 569, 1 570, 1 571, 1 572, 1 573, 1 574, 1 575, 1 576, 1 577, 1 578, 1 579, 1 580, 1 581, 1 582, 1 583, 1 584, 1 585, 1 586, 1 587, 1 588, 1 589, 1 590, 1 591, 1 592, 1 593, 1 594, 1 595, 1 596, 1 597, 1 598, 1 599, 1 600, 1 601, 1 602, 1 603, 1 604, 1 605, 1 606, 1 607, 1 608, 1 609, 1 610, 1 611, 1 612, 1 613, 1 614, 1 615, 1 616, 1 617, 1 618, 1 619, 1 620, 1 621, 1 622, 1 623, 1 624, 1 625, 1 626, 1 627, 1 628, 1 629, 1 630, 1 631, 1 632, 1 633, 1 634, 1 635, 1 636, 1 637, 1 638, 1 639, 1 640, 1 641, 1 642, 1 643, 1 644, 1 645, 1 646, 1 647, 1 648, 1 649, 1 650, 1 651, 1 652, 1 653, 1 654, 1 655, 1 656, 1 657, 1 658, 1 659, 1 660, 1 661, 1 662, 1 663, 1 664, 1 665, 1 666, 1 667, 1 668, 1 669, 1 670, 1 671, 1 672, 1 673, 1 674, 1 675, 1 676, 1 677, 1 678, 1 679, 1 680, 1 681, 1 682, 1 683, 1 684, 1 685, 1 686, 1 687, 1 688, 1 689, 1 690, 1 691, 1 692, 1 693, 1 694, 1 695, 1 696, 1 697, 1 698, 1 699, 1 700, 1 701, 1 702, 1 703, 1 704, 1 705, 1 706, 1 707, 1 708, 1 709, 1 710, 1 711, 1 712, 1 713, 1 714, 1 715, 1 716, 1 717, 1 718, 1 719, 1 720, 1 721, 1 722, 1 723, 1 724, 1 725, 1 726, 1 727, 1 728, 1 729, 1 730, 1 731, 1 732, 1 733, 1 734, 1 735, 1 736, 1 737, 1 738, 1 739, 1 740, 1 741, 1 742, 1 743, 1 744, 1 745, 1 746, 1 747, 1 748, 1 749, 1 750, 1 751, 1 752, 1 753, 1 754, 1 755, 1 756, 1 757, 1 758, 1 759, 1 760, 1 761, 1 762, 1 763, 1 764, 1 765, 1 766, 1 767, 1 768, 1 769, 1 770, 1 771, 1 772, 1 773, 1 774, 1 775, 1 776, 1 777, 1 778, 1 779, 1 780, 1 781, 1 782, 1 783, 1 784, 1 785, 1 786, 1 787, 1 788, 1 789, 1 790, 1 791, 1 792, 1 793, 1 794, 1 795, 1 796, 1 797, 1 798, 1 799, 1 800, 1 801, 1 802, 1 803, 1 804, 1 805, 1 806, 1 807, 1 808, 1 809, 1 810, 1 811, 1 812, 1 813, 1 814, 1 815, 1 816, 1 817, 1 818, 1 819, 1 820, 1 821, 1 822, 1 823, 1 824, 1 825, 1 826, 1 827, 1 828, 1 829, 1 830, 1 831, 1 832, 1 833, 1 834, 1 835, 1 836, 1 837, 1 838, 1 839, 1 840, 1 841, 1 842, 1 843, 1 844, 1 845, 1 846, 1 847, 1 848, 1 849, 1 850, 1 851, 1 852, 1 853, 1 854, 1 855, 1 856, 1 857, 1 858, 1 859, 1 860, 1 861, 1 862, 1 863, 1 864, 1 865, 1 866, 1 867, 1 868, 1 869, 1 870, 1 871, 1 872, 1 873, 1 874, 1 875, 1 876, 1 877, 1 878, 1 879, 1 880, 1 881, 1 882, 1 883, 1 884, 1 885, 1 886, 1 887, 1 888, 1 889, 1 890, 1 891, 1 892, 1 893, 1 894, 1 895, 1 896, 1 897, 1 898, 1 899, 1 900, 1 901, 1 902, 1 903, 1 904, 1 905, 1 906, 1 907, 1 908, 1 909, 1 910, 1 911, 1 912, 1 913, 1 914, 1 915, 1 916, 1 917, 1 918, 1 919, 1 920, 1 921, 1 922, 1 923, 1 924, 1 925, 1 926, 1 927, 1 928, 1 929, 1 930, 1 931, 1 932, 1 933, 1 934, 1 935, 1 936, 1 937, 1 938, 1 939, 1 940, 1 941, 1 942, 1 943, 1 944, 1 945, 1 946, 1 947, 1 948, 1 949, 1 950, 1 951, 1 952, 1 953, 1 954, 1 955, 1 956, 1 957, 1 958, 1 959, 1 960, 1 961, 1 962, 1 963, 1 964, 1 965, 1 966, 1 967, 1 968, 1 969, 1 970, 1 971, 1 972, 1 973, 1 974, 1 975, 1 976, 1 977, 1 978, 1 979, 1 980, 1 981, 1 982, 1 983, 1 984, 1 985, 1 986, 1 987, 1 988, 1 989, 1 990, 1 991, 1 992, 1 993, 1 994, 1 995, 1 996, 1 997, 1 998, 1 999, 1 1000, 1 1001, 1 1002, 1 1003, 1 1004, 1 1005, 1 1006, 1 1007, 1 1008, 1 1009, 1 1010, 1 1011, 1 1012, 1 1013, 1 1014, 1 1015, 1 1016, 1 1017, 1 1018, 1 1019, 1 1020, 1 1021, 1 1022, 1 1023, 1 1024, 1 1025, 1 1026, 1 1027, 1 1028, 1 1029, 1 1030, 1 1031, 1 1032, 1 1033, 1 1034, 1 1035, 1 1036, 1 1037, 1 1038, 1 1039, 1 1040, 1 1041, 1 1042, 1 1043, 1 1044, 1 1045, 1 1046, 1 1047, 1 1048, 1 1049, 1 1050, 1 1051, 1 1052, 1 1053, 1 1054, 1 1055, 1 1056, 1 1057, 1 1058, 1 1059, 1 1060, 1 1061, 1 1062, 1 1063, 1 1064, 1 1065, 1 1066, 1 1067, 1 1068, 1 1069, 1 1070, 1 1071, 1 1072, 1 1073, 1 1074, 1 1075, 1 1076, 1 1077, 1 1078, 1 1079, 1 1080, 1 1081, 1 1082, 1 1083, 1 1084, 1 1085, 1 1086, 1 1087, 1 1088, 1 1089, 1 1090, 1 1091, 1 1092, 1 1093, 1 1094, 1 1095, 1 1096, 1 1097, 1 1098, 1 1099, 1 1100, 1 1101, 1 1102, 1 1103, 1 1104, 1 1105, 1 1106, 1 1107, 1 1108, 1 1109, 1 1110, 1 1111, 1 1112, 1 1113, 1 1114, 1 1115, 1 1116, 1 1117, 1 1118, 1 1119, 1 1120, 1 1121, 1 1122, 1 1123, 1 1124, 1 1125, 1 1126, 1 1127, 1 1128, 1 1129, 1 1130, 1 1131, 1 1132, 1 1133, 1 1134, 1 1135, 1 1136, 1 1137, 1 1138, 1 1139, 1 1140, 1 1141, 1 1142, 1 1143, 1 1144, 1 1145, 1 1146, 1 1147, 1 1148, 1 1149, 1 1150, 1 1151, 1 1152, 1 1153, 1 1154, 1 1155, 1 1156, 1 1157, 1 1158, 1 1159, 1 1160, 1 1161, 1 1162, 1 1163, 1 1164, 1 1165, 1 1166, 1 1167, 1 1168, 1 1169, 1 1170, 1 1171, 1 1172, 1 1173, 1 1174, 1 1175, 1 1176, 1 1177, 1 1178, 1 1179, 1 1180, 1 1181, 1 1182, 1 1183, 1 1184, 1 1185, 1 1186, 1 1187, 1 1188, 1 1189, 1 1190, 1 1191, 1 1192, 1 1193, 1 1194, 1 1195, 1 1196, 1 1197, 1 1198, 1 1199, 1 1200, 1 1201, 1 1202, 1 1203, 1 1204, 1 1205, 1 1206, 1 1207, 1 1208, 1 1209, 1 1210, 1 1211, 1 1212, 1 1213, 1 1214, 1 1215, 1 1216, 1 1217, 1 1218, 1 1219, 1 1220, 1 1221, 1 1222, 1 1223, 1 1224, 1 1225, 1 1226, 1 1227, 1 1228, 1 1229, 1 1230, 1 1231, 1 1232, 1 1233, 1 1234, 1 1235, 1 1236, 1 1237, 1 1238, 1 1239, 1 1240, 1 1241, 1 1242, 1 1243, 1 1244, 1 1245, 1 1246, 1 1247, 1 1248, 1 1249, 1 1250, 1 1251, 1 1252, 1 1253, 1 1254, 1 1255, 1 1256, 1 1257, 1 1258, 1 1259, 1 1260, 1 1261, 1 1262, 1 1263, 1 1264, 1 1265, 1 1266, 1 1267, 1 1268, 1 1269, 1 1270, 1 1271, 1 1272, 1 1273, 1 1274, 1 1275, 1 1276, 1 1277, 1 1278, 1 1279, 1 1280, 1 1281, 1 1282, 1 1283, 1 1284, 1 1285, 1 1286, 1 1287, 1 1288, 1 1289, 1 1290, 1 1291, 1 1292, 1 1293, 1 1294, 1 1295, 1 1296, 1 1297, 1 1298, 1 1299, 1 1300, 1 1301, 1 1302, 1 1303, 1 1304, 1 1305, 1 1306, 1 1307, 1 1308, 1 1309, 1 1310, 1 1311, 1 1312, 1 1313, 1 1314, 1 1315, 1 1316, 1 1317, 1 1318, 1 1319, 1 1320, 1 1321, 1 1322, 1 1323, 1 1324, 1 1325, 1 1326, 1 1327, 1 1328, 1 1329, 1 1330, 1 1331, 1 1332, 1 1333, 1 1334, 1 1335, 1 1336, 1 1337, 1 1338, 1 1339, 1 1340, 1 1341, 1 1342, 1 1343, 1 1344, 1 1345, 1 1346, 1 1347, 1 1348, 1 1349, 1 1350, 1 1351, 1 1352, 1 1353, 1 1354, 1 1355, 1 1356, 1 1357, 1 1358, 1 1359, 1 1360, 1 1361, 1 1362, 1 1363, 1 1364, 1 1365, 1 1366, 1 1367, 1 1368, 1 1369, 1 1370, 1 1371, 1 1372, 1 1373, 1 1374, 1 1375, 1 1376, 1 1377, 1 1378, 1 1379, 1 1380, 1 1381, 1 1382, 1 1383, 1 1384, 1 1385, 1 1386, 1 1387, 1 1388, 1 1389, 1 1390, 1 1391, 1 1392, 1 1393, 1 1394, 1 1395, 1 1396, 1 1397, 1 1398, 1 1399, 1 1400, 1 1401, 1 1402, 1 1403, 1 1404, 1 1405, 1 1406, 1 1407, 1 1408, 1 1409, 1 1410, 1 1411, 1 1412, 1 1413, 1 1414, 1 1415, 1 1416, 1 1417, 1 1418, 1 1419, 1 1420, 1 1421, 1 1422, 1 1423, 1 1424, 1 1425, 1 1426, 1 1427, 1 1428, 1 1429, 1 1430, 1 1431, 1 1432, 1 1433, 1 1434, 1 1435, 1 1436, 1 1437, 1 1438, 1 1439, 1 1440, 1 1441, 1 1442, 1 1443, 1 1444, 1 1445, 1 1446, 1 1447, 1 1448, 1 1449, 1 1450, 1 1451, 1 1452, 1 1453, 1 1454, 1 1455, 1 1456, 1 1457, 1 1458, 1 1459, 1 1460, 1 1461, 1 1462, 1 1463, 1 1464, 1 1465, 1 1466, 1 1467, 1 1468, 1 1469, 1 1470, 1 1471, 1 1472, 1 1473, 1 1474, 1 1475, 1 1476, 1 1477, 1 1478, 1 1479, 1 1480, 1 1481, 1 1482, 1 1483, 1 1484, 1 1485, 1 1486, 1 1487, 1 1488, 1 1489, 1 1490, 1 1491, 1 1492, 1 1493, 1 1494, 1 1495, 1 1496, 1 1497, 1 1498, 1 1499, 1 1500, 1 1501, 1 1502, 1 1503, 1 1504, 1 1505, 1 1506, 1 1507, 1 1508, 1 1509, 1 1510, 1 1511, 1 1512, 1 1513, 1 1514, 1 1515, 1 1516, 1 1517, 1 1518, 1 1519, 1 1520, 1 1521, 1 1522, 1 1523, 1 1524, 1 1525, 1 1526, 1 1527, 1 1528, 1 1529, 1 1530, 1 1531, 1 1532, 1 1533, 1 1534, 1 1535, 1 1536, 1 1537, 1 1538, 1 1539, 1 1540, 1 1541, 1 1542, 1 1543, 1 1544, 1 1545, 1 1546, 1 1547, 1 1548, 1 1549, 1 1550, 1 1551, 1 1552, 1 1553, 1 1554, 1 1555, 1 1556, 1 1557, 1 1558, 1 1559, 1 1560, 1 1561, 1 1562, 1 1563, 1 1564, 1 1565, 1 1566, 1 1567, 1 1568, 1 1569, 1 1570, 1 1571, 1 1572, 1 1573, 1 1574, 1 1575, 1 1576, 1 1577, 1 1578, 1 1579, 1 1580, 1 1581, 1 1582, 1 1583, 1 1584, 1 1585, 1 1586, 1 1587, 1 1588, 1 1589, 1 1590, 1 1591, 1 1592, 1 1593, 1 1594, 1 1595, 1 1596, 1 1597, 1 1598, 1 1599, 1 1600, 1 1601, 1 1602, 1 1603, 1 1604, 1 1605, 1 1606, 1 1607, 1 1608, 1 1609, 1 1610, 1 1611, 1 1612, 1 1613, 1 1614, 1 1615, 1 1616, 1 1617, 1 1618, 1 1619, 1 1620, 1 1621, 1 1622, 1 1623, 1 1624, 1 1625, 1 1626,
```

```

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected: boolean } {
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true, connected: true };
 const heapOk = edges.every((e) => e.a.grade > e.b.grade);
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (parentCount.get(pid(e.b)) || 0) + 1);
 const singleParent = [...parentCount.values()].every(v => v === 1);
 // связность: компоненты через union-find
 const comp = new Map<string, string>(points.map((p) => [p, p]));
 const find = (x: string): string => {
 while (comp.get(x) !== x) {
 comp.set(x, comp.get(x)!);
 x = comp.get(x)!;
 }
 return x;
 };
 for (const e of edges) {
 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
 }
 const comps = new Set(points.map((p) => find(pid(p))));
 const connected = comps.size === 1;
 const countOk = edges.length === n - 1;
 // BST: дети по x, inorder не убывает; корень один; э.
 const children = new Map<string, Pair[]>();
 for (const e of edges) children.set(pid(e.a), [...(children.get(pid(e.a)) || []), e.b]);
 const roots = points.filter((p) => !parentCount.has(pid(p)));
 let bstOk = roots.length === 1;
 if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 }
 };
 }
}

```

```

for (const e of edges) parentOf.set(pid(e.b), e);
const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
if (!roots.length && edges.length) return null;
const nodes = new Map<string, TNode>();
const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
});
const rootPair = roots.length ? roots[0].a : edges.length > 0 ? edges[0].a : null;
if (!rootPair) return null;
const build = (p: Pair): TNode => {
 const node = make(p);
 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p));
 if (kids.length > 2) return node; // невалидно – по
 kids.sort((u, v) => u.key - v.key);
 const left = kids.filter((k) => k.key <= p.key); //
 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node;
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
};
return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const [phase, setPhase] = useState<BuildPhase>("sort");
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d");
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [sortCost, setSortCost] = useState<string | null>();

```

```

 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>();
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>();
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair, canonEdges));
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(connEdges, canonEdges) : null));

const attemptConnect = (x: Pair, y: Pair) => {

```





```

 onClick={() => startConnect("left-right")}
 className={`px-3 py-1.5 rounded-lg text-white
 "bg-emerald-600 text-white" : "bg-slate-700 text-white"}
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-white
 "go-600 text-white" : "bg-indigo-500/80 text-white"}
 >
 соединить сам
 </button>
 </>
)}
</div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : phase === "sort" ? sortEdges : []}
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-slate-800/50 p-2"
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} пар)
 </>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: cl

```

```

 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно <b
 </>
 })

{phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
 white disabled:opacity-30">↶ Убрать линию</
 <button onClick={resetGame} disabled={!gameE
 ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
 te disabled:opacity-30"> Показать решение<
 </div>
)}
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-
 <h4 className="text-sm font-bold text-emerald
 <TreeView root={gameTree} subtitle="Слева-нап
 ществует." />
 </div>
)}

{/* Дерево */}
{order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={userTree} subtitle="Слева-нап
 </div>
)}

```

```

 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
 </div>

 <div className="text-xs text-slate-300 bg-slate-900
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2
 <div className="bg-sky-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-sky-300
 <TreeView root={lTree} width={480} height={
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-rose-:
 <TreeView root={rTree} width={480} height={
 </div>
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900
 Как это работает на самом деле: идём с
 деревом), отдаётся вместе с веткой – д
 </div>
 </div>
);
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {

```

```

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-3">
 <h5 className="text-sm font-bold text-emerald-500">Грамотно про «слияние»:</h5>
 <TreeView root={merged.root} subtitle="merge">
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900/20 p-3">
 Грамотно про «слияние»: у корней сравнения
 в (то, что сохраняет порядок x) отправляется ,
 </div>
 </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const res = useMemo(() => eraseNode(buildTree(POINTS), key), [key]);
 const steps = res.trace;
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId)
 return ids;
 }, [steps, idx]);
 const pathSet = new Set(pathIds);

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Удалить:
 <select value={key} onChange={(e) => { setKey(Number(e.target.value)) }}>

```

```

 </div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null)
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 walk(full, 1);
 return arr;
 }, [full]);
 const nodeCount = slots.filter(Boolean).length;

 return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="bg-slate-700 text-white ext-white" : "bg-slate-700 text-slate-300"}`>
 <button onClick={() => setPick("treap")} className="bg-slate-700 text-white" : "bg-slate-700 text-slate-300">
 </div>

 {pick === "heap" ? (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">
 4>

```

```

 ивым: слои дырявые, и «адрес» ребёнка посчита
 </div>
 </div>
);
};

```

/\* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) ===== \*/

```

const SearchMode: React.FC = () => {
 // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО пр
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },
 { key: 9, grade: 3 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 7, grade: 2 },
 { key: 1, grade: -4 },
];
 const target = 7;
 const sortedByKey = [...shuffled].sort((a, b) => a.key
 const [stage, setStage] = useState<0 | 1 | 2>(0); //
 const [bidx, setBidx] = useState(0);

 // шаги бинарного поиска на отсортированном массиве д
 const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number;
 let lo = 0, hi = sortedByKey.length - 1;
 while (lo <= hi) {
 const mid = Math.floor((lo + hi) / 2);
 if (sortedByKey[mid].key === target) { steps.push(
 нашли за ${steps.length + 1} сравнение(й).` });
 if (sortedByKey[mid].key < target) { steps.push({
 } – вся левая половина выброшена.` }); lo = mid
 else { steps.push({ lo, hi, mid, found: false, no
 ` }); hi = mid - 1; }
 }
 }, [sortedByKey, target]);

```

```

 </div>
);
 }}}
</div>

<div className="text-xs text-slate-300 bg-slate-900" >
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на
 ю половину... в которой лежала цель. На
 реставил: массив как был в беспорядке, та
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал,
 сравнения на всех. «Отсортирую бинарный
</div>

<div className="text-xs text-slate-400 bg-slate-900" >
 Есть ли разница, чем сортировать? Сравн
 няя граница, каждое сравнение даёт 1 бит. Но
 ля маленьких целых у (вкладка «Собрать», кноп
 задать порядок соединения.
</div>
</div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);
 const [sel, setSel] = useState<Pair | null>(null);
 const [hover, setHover] = useState<Pair | null>(null);
 const [msg, setMsg] = useState<string>("Кликни точку-1");

 const v = gameVerdict(edges, POINTS);

```



```

<svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} class=
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" s
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height=
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y

 {edges.map((e, i) => {
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399"
 strokeWidth={i === edges.length - 1 ? 3.5
 className="transition-all duration-300"
 />
 }}}

 {POINTS.map((p) => {
 const isSel = sel && sameP(sel, p);
 const isHover = hover && sameP(hover, p) &&
 return {
 <g key={`gp-${p.key}:${p.grade}`} onClick=
 l)} className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)}
 <circle cx={PX(p.key)} cy={PY(p.grade)}
 "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
 transition-all duration-200" />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5
 space">{p.key}</text>
 <text x={PX(p.key)} y={PY(p.grade) + 24
 }</text>
 </g>
 });
 }}}

```

```
export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пап
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
 </div>

 {tab === "build" && <BuildMode />}
 {tab === "split" && <SplitMode />}
 {tab === "merge" && <MergeMode />}
 {tab === "erase" && <EraseMode />}
 {tab === "layers" && <LayersMode />}
 {tab === "search" && <SearchMode />}
 {tab === "game" && <GameMode />}
```

Универсальное пособие • полный исходный код

```

import { SparseTableViz } from "../visualizers/SparseTab
import { TreapBuildViz } from "../TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы – куча сама держит лучшую
 Component: HeapViz,
 },
 "segment-trees": {
```

```

"scc-kosaraju": { title: "Компоненты сильной связности", Component: SCC,
"graph-articulation": { title: "Точки сочленения", Component: GraphArticulation,
"bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCode,
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPathVsCycle,
"planarity-euler-formula": { title: "Планарность: К5 и K3,3", Component: PlanarityEulerFormula,
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => {
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
 },
},
"bellman-ford": {
 title: "Форд-Беллман",
 Component: () => {
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
 },
},
floyd: {
 title: "Флойд-Уоршелл",
 Component: () => {
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
 },
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgo,
"mst-kruskal": { title: "Краскал и DSU", Component: Kruskal,
"mst-prima": { title: "Прим", Component: KruskalSimul,
"mst-boruvka": { title: "Борувка", Component: KruskalSimul,
"string-kmp": { title: "Префикс-функция (КМП)", Component: StringKMP,

```

```

 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0,
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1,
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2,
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3,
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4,
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2,
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3,
 // Ветка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1,
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2,
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3,
 = 2 (HE)
 }
 }
 }
 }
 }
 }
 }
 }
 }
 };

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};

```

```

 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен!');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label})`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лосось доволен!`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} (поток по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}))`;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 }
 }

```

```

 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди - просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на De
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
 а-разведчик плывет в ROOT и ищет ветку по "E".
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
 },

```

```

 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) – КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. По
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
],
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNode];

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border
 {/* Шапка и переключатель режимов */}

```



```
</div>
```

```
{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с
 ar 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow" >
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white fon
 : 'bg-slate-800 text-slate-400
 }`}
 >
 {step
 {idx === trainStep && <span classNa
 </button>
)}}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
```

```

 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
к родителю}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/span>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold ml
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>
</div>

```

```

)))
 {currentIndex >= textToScan.length && (

 ГОТОВО ✓

)}
 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-mono text-sm
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 }`>
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4" >
 <div>
 <h5 className="text-white font-bold mb-3" >
 ✂ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-lg" >
 <div className={`p-1.5 rounded flex items-center border border-blue-500 text-blue-300 font-bold`} >
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && инициализация}
 </div>
 <div className={`p-1.5 rounded flex items-center border border-rose-500 text-rose-300 font-bold an

```

```

<div className="absolute top-0 left-1/2 -transl
 <div className="w-8 bg-blue-400 h-full animat
 <div className="w-12 bg-blue-300 h-full anima
 <div className="w-6 bg-blue-500 h-full animat
 </div>
 </div>

```

```

<div className="flex items-center justify-between
 <h5 className="text-white font-bold text-base
 Ветвящийся синий водопад (Бо
 </h5>
 <div className="flex flex-wrap gap-4 text-xs"
 <span className="flex items-center gap-1 te
 <span className="w-3 h-0.5 bg-blue-500 in

 <span className="flex items-center gap-1 te
 <span className="w-3 h-0.5 border-t borde

 </div>
</div>

```

```

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-s
 <svg viewBox="0 0 800 520" className="w-full

```

```

 {/* Горизонтальные линии и подписи уровней */}
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень'
 { depth: 1, y: 160, label: 'Depth 1 (H, S'
 { depth: 2, y: 260, label: 'Depth 2 (HE, I'
 { depth: 3, y: 360, label: 'Depth 3 (HER,'
 { depth: 4, y: 460, label: 'Depth 4 (HERS'
]}.map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}

```

```

 />
 { /* Символ перехода */ }
 <circle cx={{(fromNode.x + toNode.x)
ined ? (isMatchBranch ? '#10b981' : '#f43f5
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2 +
 fill={isTrainingExamined ? (isMat
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 >
 {char}
 </text>

 { /* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
 {isTrainingExamined && (
 <g transform={`translate(${(fromN
.y) / 2}}`}>
 <rect x="-15" y="-12" width="30
strokeWidth="2" />
 <text x="0" y="5" fontSize="14"
 {isMatchBranch ? ' ' : ' '}
 </text>
 </g>
)}
 </g>
)};
 });
 }}}

 { /* Рендер fail-ссылок (пунктирные водопадн
 {Object.values(WATERFALL_NODES).map((node) :
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.f

 // В режиме обучения показываем fail-ссыл
 // Найдем индекс шага, на котором обработ

```

```

const isTargetChild = activeMode === 'transition-all'
const hasWords = node.words.length > 0;

return (
 <g key={`node-${node.id}`} className="node"
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={radius}
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={radius}
)}

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24px' : '12px'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : 'white'}
 stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : 'white'}
 strokeWidth={isCurrent || isTargetChild ? 2 : 1}
 className="transition-all duration-300"
 />

 {/* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 fontSize={isCurrent || isTargetChild ? 14 : 12}
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме обзора */}
 {isTargetChild && (

```

```

 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezier(0.25, 0.8, 0.25, 0.75)' }}
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' : ' ♀ '}
 </text>
 </g>

 </svg>
</div>
</div>

{/* Список найденных совпадений (только в режиме поиска) */}
{activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded-lg">
 <h5 className="text-white font-bold mb-3 text-sm">
 ⚠ Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic">
 Пока ничего не поймали. Запустите шаги симуляции.
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-800 text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-400" />
 {match.word}

 Индекс {match.index}

 </div>
))}
 </div>
)}
 </div>
)}

```

```

const cellBase =
 "flex items-center justify-center rounded-md font-mono"
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border"
 <div className="flex gap-2 mb-4 overflow-x-auto no-wrap"
 {{
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
 }.map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
 }`}
 >
 {label}
 </button>
)))
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v] + v],
 []);
};

```



```

onClick={() => setRange((r) => (i < r
className={`${cellBase} cursor-pointe
inCover
 ? "bg-indigo-500 text-white ring-
 : inRange
 ? "bg-indigo-900/70 text-indigo
 : "bg-slate-800 text-slate-300
 `}
 title={`a[${i}] = ${v}`}
>
 {v}
</div>
);
}}
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" &&
 const isL = i === range.l;
 const isR = i === range.r + 1;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 className={`${cellBase} cursor-help $
 active
 ? "bg-emerald-500 text-white ring
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-30
 `}
 >
 title={`P[${i}] = сумма a[0..${i} - 1]

```

```

 <p className="text-slate-500">
 Наведите курсор на любое число: у <b className="text-slate-500">у
 у <b className="text-indigo-400">a – ку,
 </p>
)}
 </div>

 { /* Запрос */ }
 <div className="bg-slate-900 border border-indigo-400">
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base text-slate-500">
 $\text{sum}(a[\text{range.l}..\text{range.r}]) = P[\text{range.r} + 1]$
 {pref[range.r + 1]}
 {pref[range.l]}
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за $O(1)$ по
 </p>
 </div>
 </div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array.from(
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1]
 }
 }
)
 }, [A2])
}
```

```

 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }` }
 >
 {v}
 </div>
);
}}}
</div>
)}}
</div>
</div>

```

```

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {P.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hov
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
 return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={` ${cellBase} cursor-he
 isHover ? "bg-amber-500 text-wh

```

```
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
 </div>
</div>
);
};
```

---

ФАЙЛ 72/83: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 709 | РАЗМ: 10.5 КБ

---

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << (j - 1))][j - 1]);
 }
}

// Data for 2D Matrix (Sparse Table 2D)
```



```

<div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1)}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1)}
 className="p-2 bg-indigo-600 hover:bg-indigo-500 disabled:opacity-50"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 >
 <RotateCcw size={20} />
 </button>
</div>
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <div className="text-[11px] uppercase tracking-wider">
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto">
 {arr1D.map((v, idx) => {
 const inCover = !!covered && idx >= covered - 1;
 return (
 <div key={idx} className="flex flex-col items-center justify-center gap-1">
 <div
 className={`flex items-center justify-center gap-1 w-[44px] h-[44px] text-[11px] font-medium ${
 inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-400"
 }`}
 >
 {v}
 </div>
 </div>
);
 })}
 </div>
 </div>
</div>

```

```

 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-rap">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-rap">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-rap">
 2^3 (L=8)
 </div>

 {/* Matrix */}
 {Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-monospace">
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const isVisible =
 isValid &&
 (j === 0 ||
 computeSteps.findIndex((s) => s.i === i && s.j === j) <= step);

 // Active computing state
 let isActive = false;
 let isSrc1 = false;
 let isSrc2 = false;

 if (step > 0 && step <= maxStep) {
 const currentStep = computeSteps[step];
 if (currentStep.i === i && currentStep.j === j) {
 isActive = true;
 }
 }
 })}
 </React.Fragment>
 })}

```

```

 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"
)` }
 >
 {st1D[i][j]}
 </motion.div>
)}

 { /* Arrows visualization */
 {(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc1 ? "0 0" : "50 50"}`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#f59e00"}
 strokeWidth="3"
 />
 </motion.svg>
)}
 </div>
);
}}
</React.Fragment>
)}}

```



```

 </p>
)}
 </div>
 </div>
);
};

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number}>({ r: 0, c: 0 });

 const L = 1 << k;
 const activeCell = locked || hover;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scrollable">
 <h3 className="text-xl font-bold text-indigo-400">Задача
 <p className="text-sm text-slate-400 mb-6 text-wrap">
 Для наглядности показываем только <b className="text-slate-400">

 иц!
 </p>

 <div className="flex bg-slate-950 p-1 rounded-l-1">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(null); setLocked(null); }}
 className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-900`}
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
 </button>
))}
 </div>
 </div>
 </div>
);
};

```

```

 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCell;
 const isLeft = c < activeCell;

 if (isTop && isLeft) {
 highlightClass = 'bg-rose';
 } else if (isTop && !isLeft) {
 highlightClass = 'bg-blue';
 } else if (!isTop && isLeft) {
 highlightClass = 'bg-emerald';
 } else {
 highlightClass = 'bg-amber';
 }
 zIndex = 'z-10';
 }
 }

 return (
 <div
 key={idx}
 onMouseEnter={() => { if(isCurr) setCurr(r, c); }}
 onMouseLeave={() => { if(isCurr) setCurr(r, c); }}
 onClick={() => {
 if(isCurr) {
 if(locked && locked.r === r && locked.c === c) {
 return;
 }
 setLocked({r, c});
 }
 }}
 className={`flex items-center justify-center gap-2`}
 style={{
 width: sCurr ? `w-[clamp(24px,6.5vw,38px)]` : `w-[clamp(20px,5.2vw,30px)]`,
 height: sCurr ? `h-[clamp(24px,6.5vw,38px)]` : `h-[clamp(20px,5.2vw,30px)]`,
 text-align: center,
 font-size: `font-size-[clamp(8px,2vw,12px)]`,
 font-weight: `font-weight-[clamp(400,1.2,600)]`,
 color: `color-[${ST2D[r][c][step][step]}]`,
 }}
 >
 {ST2D[r][c][step][step]}
 </div>
);
}

```

```

 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>


```

```

 <span className="text-emerg
="text-amber-300">1][k-<span classNa
span>


```

```

 <span className="text-amber
white">c + pL][k-<span className="te
ame="text-slate-500">// Правый Низ<b
{'}}'));
</div>

```

```

 {activeCell ? (
 <div className="bg-slate-950 p-5 rounded
.2)] mt-2 transition-all">
 <p className="text-slate-300 mb-3 tr
 Пример для ячейки:
 {locked && <span className="text
ded border border-rose-500/20"><Lock size={
 </p>
 <p className="text-indigo-300 borde
 <span className="font-bold text-w
 }}[k]][k]

 </p>

```

```

 <div className="space-y-2 pt-3 pl-2
 {(() => {
 const prevL = 1 << (k - 1);
 return (
 <>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c][k-1][k-1]}
 </div>
 <div className="flex ite

```

```

 </div>
 </div>
);
 };
};

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;
 const W = c2 - c1 + 1;
 const kx = Math.floor(Math.log2(H));
 const ky = Math.floor(Math.log2(W));
 const Lx = 1 << kx;
 const Ly = 1 << ky;

 const matRows = 8 - Lx + 1;
 const matCols = 8 - Ly + 1;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900">
 <h3 className="text-xl font-bold text-rose-500">Задача 1
 <p className="text-slate-400 text-[13px] font-normal">
 (r1, c1 - левый верх; r2, c2 - правый низ).</p>

 <div className="flex flex-col items-center">
 <div className="grid grid-cols-[repeat(8, 1fr)] gap-1 shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_, idx) => {
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;

 return (
 <div key={idx} className={`w-8 h-8 flex items-center justify-center
 ${isInQuery ? 'bg-rose-500' : 'bg-slate-900'}
 text-[10px] font-normal ${isInQuery ? 'text-slate-900' : 'text-rose-500'}
 transition-colors duration-200`}>
 {isInQuery ? 1 : 0}
 </div>
);
 })}
 </div>
 </div>
 </div>
 </div>
);
};

```

```


 <div className="flex gap-2">
 <input type="number" min={Lx} max={Lx} value={Lx} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 <input type="number" min={Ly} max={Ly} value={Ly} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 </div>
 </label>
 <label className="flex items-center gap-2">

 <div className="flex gap-2">
 <input type="number" min={Lx} max={Lx} value={Lx} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 <input type="number" min={Ly} max={Ly} value={Ly} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 </div>
 </label>
</div>
</div>
</div>
</div>

<div className="flex-1 bg-slate-900 p-6 rounded-2xl">
 <h3 className="text-xl font-bold text-emerald-400">Задача 1</h3>
 <p className="text-sm font-sans text-slate-300">
 Мы знаем, что максимальная степень дв
 rounded text-white">{Lx}. Аналогично
 te">{Ly}.
 </p>
 <p className="text-sm font-sans text-slate-300">
 Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0f1e] px-1 rounded border-1 border-rose"
 роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0f1e] rounded-xl p-4 text-slate-300 shadow-inner overflow-x-auto">
 <div className="absolute right-3 top-3">
 // 1.

```

```

</h4>
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}>
 {Array.from({length: matRows * ma
 const r = Math.floor(idx / ma
 const c = idx % matCols;
 let isTL = r === r1 && c ===
 let isTR = r === r1 && c ===
 let isBL = r === r2 - Lx + 1
 let isBR = r === r2 - Lx + 1

 let color = "bg-slate-800 tex
 let txt = ST2D[r][c][kx][ky].

 let badge = null;
 if (isTL) { color = "bg-rose-!
0 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-l
ue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-
-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-
mber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={
tion-all duration-300 relative shrink-0 ${c
 {txt}
 {badge && <div classNam
r-slate-700 text-white px-1 font-bold round
 </div>
)
 }}}}
</div>
</div>
</div>

<div className="mt-auto bg-slate-950 fon

```

```

 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 const aside = document.querySelector('aside');
 const headerMain = document.querySelector('header.main');
 const questionsContainer = document.getElementById('questions-container');
 const wtfContainer = document.getElementById('wtf-container');

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname);
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
 }
 }

 function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 }

```

```

 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add("-translate-x-100%");
 if (drawerOverlay) {
 drawerOverlay.classList.add("opacity-100");
 setTimeout(() => drawerOverlay.classList.add("opacity-100"), 100);
 }
 document.body.style.overflow = "hidden";
 }
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => {
 toggleDrawer();
 });
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => {
 toggleDrawer();
 });
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => {
 toggleDrawer();
 });
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains("open")) {
 toggleDrawer();
 }
 });
});

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

const observer = new IntersectionObserver((entries) => {
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 toggleDrawer();
 }
 });
});

```



```

<script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 }
 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?');
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospace; }
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; text-align: center;
 color: white; overflow: hidden; flex-shrink: 0;
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #000;
 font-family: 'Arial Black', Impact, sans-serif;
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute;
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #000;
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
 .uno-card.mini { width: 45px !important; height: 60px !important; }
 .uno-card.mini::before, .uno-card.mini::after {
 font-size: 0.8em;
 }

 /* Global formula scroll settings */
 .formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
 }
 .formula-scroll::-webkit-scrollbar {
 width: 0;
 }

```

```

 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card-
 .card-green { background-color: #339e35; } .card-
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-light
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

```

```

 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
 </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Режим
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal"
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf"
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark"
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light"
 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook"
 transition-colors">Заметки</button>
 </div>
 </div>
 </div>

 <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
 <svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" markerHeight="6"
 orient="vertical"
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d" stroke="black" stroke-width="1" />
 </marker>
 </defs>
 </svg>

```

```

 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20" />
 <rect x="5" y="40" width="40" height="20" />
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30" />
 <circle cx="75" cy="30" r="4" fill="#333" />
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55" />
 <circle cx="25" cy="50" r="15" fill="#f00" />
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="10" />
 <rect x="3" y="5" width="14" height="5" />
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20" />
 <rect x="6" y="-5" width="8" height="6" />
 <rect x="26" y="-5" width="8" height="6" />
 <line x1="0" y1="2" x2="40" y2="2" stroke="black" />
 <text x="20" y="14" fill="#475569" font-weight="bold">
 ЛЕГО
 </text>
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed top-0 right-0
 0/50 z-50 hover:bg-blue-500 transition-transform duration-300" type="button">
 <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6" fill="none" stroke="black" stroke-width="2">
 <path stroke-linecap="round" stroke-linejoin="round" d="M4 6h16" />
 <path stroke-linecap="round" stroke-linejoin="round" d="M4 12h16" />
 <path stroke-linecap="round" stroke-linejoin="round" d="M4 18h16" />
 </svg>
</button>

<!-- Затемнение фона на мобилках при открытом меню -->
<div id="drawer-overlay" class="lg:hidden fixed inset-0 bg-black bg-opacity-50" style="display: none;">
 </div>

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-wrap">

```

er-indigo-500 pl-3 font-bold text-indigo-300">2. Комплексные числа

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#section-2-1" class="b

<a href="#section-2-2" class="b

<a href="#section-2-3" class="b

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-3" class="

er-teal-500 pl-3 font-bold text-teal-300">

3. Тригонометрическая форма

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#section-3-1" class="b

<a href="#section-3-2" class="b

<a href="#section-3-3" class="b

<a href="#section-3-4" class="b

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-4" class="

er-amber-500 pl-3 font-bold text-amber-300">

4. Степени и корни

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#q4-def" class="block l

<a href="#q4-moivre" class="blo

<a href="#q4-root" class="block

<a href="#q4-count" class="bloc

```
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">
```

```
 7. Взаимно простые числа
```

```

```

```
 </summary>
```

```
 <div class="pl-6 space-y-2 text-pink-300">
```

```

```

```

```

```

```

```

```

```

```

```
 </div>
```

```
</details>
```

```
<details class="group">
```

```
 <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
```

```

```

```
 8. Сравнимость и Поля Вейля
```

```

```

```
 </summary>
```

```
 <div class="pl-6 space-y-2 text-violet-300">
```

```

```

```

```

```

```

```

```

```

```

```

```

```

```

```
 </div>
```

```
</details>
```

```
<details class="group">
```

```
 <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
```

```

```

```
 9. Кольцо многочленов
```

```

 <a href="#q11-3" class="blo
ПИРАЛЬ)
 <a href="#q11-4" class="blo
вращается!)
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-12" clas
der-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q12-1" class="blo
 <a href="#q12-2" class="blo
 <a href="#q12-3" class="blo
еление)
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-13" clas
der-emerald-500 pl-3 font-bold text-emerald
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q13-1" class="blo
 <a href="#q13-2" class="blo
 <a href="#q13-3" class="blo
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
```

```


 </div>
</details>

<div class="mt-4 mb-2 text-xs font-medium">

-500 pl-3 text-slate-400">

-500 pl-3 text-slate-400">

ald-500 pl-3 text-slate-400">

-500 pl-3 text-slate-400">

go-500 pl-3 text-slate-400">

sia-500 pl-3 text-slate-400">

- red-500 pl-3 mt-4 text-red-500 font-bold mt-4">
 ▲ Ликбез: Алгоритмы выживания

</nav>
```



```

 if (a.href.endsWith('#')) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details element if the corresponding link is clicked
 if (correspondingLink) {
 const details = document.querySelector('details');
 if (details && !details.open) {
 document.querySelector('details').open = true;
 if (otherDetails) {
 otherDetails.open = false;
 }
 }
 }
}

});

}, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});
});

function fallbackModal(contentRaw, ...args) {
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';

```

```

btnContainer.style.gap = '10px'
btnContainer.style.marginTop =

const copyBtn = document.createElement('button')
copyBtn.innerText = 'Копировать'
copyBtn.style.padding = '10px'
copyBtn.style.backgroundColor = 'white'
copyBtn.style.color = 'black'
copyBtn.style.border = 'none'
copyBtn.style.borderRadius = '5px'
copyBtn.style.cursor = 'pointer'
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value)
 copyBtn.innerText = 'Скопировано'
 } else {
 textarea.select()
 document.execCommand('copy')
 copyBtn.innerText = 'Скопировано'
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000)
};

const closeBtn = document.createElement('button')
closeBtn.innerText = 'Заккрыть'
closeBtn.style.padding = '10px'
closeBtn.style.backgroundColor = 'white'
closeBtn.style.color = 'black'
closeBtn.style.border = 'none'
closeBtn.style.borderRadius = '5px'
closeBtn.style.cursor = 'pointer'
closeBtn.onclick = () => modal.close()

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(copyBtn);
 btnContainer.appendChild(closeBtn);
}

```

```

 document.body.appendChild(link)
 link.click();
 document.body.removeChild(link)
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate');

 document.getElementById('theme-dark')
 document.getElementById('theme-light')
 document.getElementById('theme-style')

 let oldStyle = document.getElementById('theme-style')
 if (oldStyle) oldStyle.remove()
 const style = document.createElement('style')
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900')
 document.getElementById('theme-dark').style.display = 'block'
 } else if (theme === 'light') {
 body.classList.add('bg-white')
 document.getElementById('theme-light').style.display = 'block'
 style.innerHTML = `
 body.bg-white section h1 { color: #000000; }
 body.bg-white main { color: #000000; }
 body.bg-white main p { color: #000000; }
 /* Universal text color */
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white [class*=text-slate-900] { color: #000000; }

 /* Specific backgrounds
 `
 }
 }

```

```

 body.bg-white .text-lime
 body.bg-white .text-blue
 body.bg-white .text-green
 body.bg-white .text-indigo
 `;
 } else if (theme === 'terminal') {
 body.classList.add('bg-black');
 document.getElementById('theme')
 .style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: transparent;
 }
 body.bg-black section header {
 color: #4ade80 !important;
 }
 body.bg-black button {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace;
 }
 body.bg-black svg path,
 body.bg-black section header {
 stroke: #22c55e !important;
 fill: transparent !important;
 }
 `;
 }

 document.head.appendChild(style);
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4">
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-500">

```

```

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16" style="background-color: #1a2b3c; color: white;">
 <header class="text-center mb-12 border-b border-white pb-4" style="border-bottom: 1px solid white; padding-bottom: 4px;">
 <h1 class="text-4xl md:text-5xl font-bl" style="font-size: 2em; margin: 0;">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg" style="color: #a0b0c0; font-style: italic; font-size: 1.2em; margin-top: 10px;">
 ...
 </p>

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center" style="display: none; margin-top: 20px;">
 <h2 class="font-bold text-xl mb-4" style="font-weight: bold; font-size: 1.2em; margin-bottom: 10px;">
 Содержание
 </h2>
 <ul class="text-base space-y-2" style="list-style-type: none; padding: 0; margin: 0;">
 1. Алгебра
 2. Коммутативная алгебра
 3. Топология
 4. Статистика
 5. Группы
 6. Декартовы координаты
 7. Алгебраические структуры
 8. Кривые
 9. Координаты
 10. Матрицы
 11. Линейные пространства
 12. Преобразования
 13. Линейные операторы
 14. Линейные функции
 15. Комплексные числа
 17. Матрицы
 18. Группы
 19. Алгебраические структуры
 20. Декартовы координаты
 21. Топология
 22. Коммутативная алгебра

 </div>
 </main>

```

```


 <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 3: НОД и НОК / Алгоритм Ев
 <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структур
 <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM

```

b-2 border-b border-emerald-500/30 pb-2">Кл

[\[\\[\\\[\\\\[\\\\\[<!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ  
<div class="col-span-1 md:col-span-1  
border-b border-red-500/30 pb-2">Кластер 4:\\\\\]\\\\\(#question-22\\\\\)\\\\]\\\\(#question-21\\\\)\\\]\\\(#question-20\\\)\\]\\(#question-19\\)\]\(#question-18\)](#question-17)

```

 <span class="text-w
ово.

</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-fuch">
 <p class="text-sm text-slat">
 <ul class="list-disc pl-5 sp">
 <span class="text-w
i>
 <span class="text-w
т.д.).
 <span class="text-w
что так и задумано: "Ой, противоречие, теор

</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-eme">
 <p class="text-sm text-slat">
 <ul class="list-disc pl-5 sp">
 <span class="text-w
, r_2$).
 <span class="text-w
 <span class="text-w
счет ограничений (размер остатка) докажи, ч

</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-yel">
 <p class="text-sm text-slat">
 <ul class="list-disc pl-5 sp">
 <span class="text-w

```



$\$c\$$ . Пока получается 0 — корень жив. Как то  
корня минус один.</p>

</div>

<div class="bg-slate-800 p-

<h4 class="text-red-400

<p class="text-sm text-

<p class="text-xs font-

ициентов (они делятся на  $\$p\$$ ), кроме старше  
зложить.</p>

</div>

<div class="bg-slate-800 p-

<h4 class="text-red-400

<p class="text-sm text-

<p class="text-xs font-

. Чаще всего после этого Фейс-контроль  $\$p\$$  .

</div>

</div>

<h2 class="text-2xl font-bold b

(Правила экстрасенса)</h2>

<p class="text-sm text-slate-40

тношения), а ты их не помнишь. Юзай базовые  
нейность) для делимости.</p>

<ul class="space-y-4 pb-10">

<li class="bg-slate-800/50 p

<span class="text-2xl">

<div>

<h4 class="text-cyan

<p class="text-sm t

<code class="text-x

</div>

</li>

<li class="bg-slate-800/50 p

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 75/83: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 76/83: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили зашиты внутрь, интерактив
 * его можно открыть с флешки, отправить в мессенджер и т.д.
 * Живые React-визуализации в статику не переносятся — вместо них
 * подставляется заметка со ссылкой на онлайн-версию, а для элементов
 * которые в приложении всплывают по наведению, собираю их в массив
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при отсутствии — fallback
```

```
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`;
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на
```

```
 * Возвращает готовый HTML главы и список найденных терминов
```

```
 */
```

```
function prepareBody(html: string): { body: string; terms: string[] } {
 const host = document.createElement("div");
 host.innerHTML = html;
```

```
 try {
```

```
 annotateTerms(host);
```

```
 } catch {
```

```
 /* если браузер не осилил регулярку – выгружаем текст как есть */
```

```
 }
```

```
 const ids: string[] = [];
```

```
 host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
```

```
 const id = el.dataset.termId;
```

```
 if (!id) return;
```

```
 if (!ids.includes(id)) ids.push(id);
```

```
 const link = document.createElement("a");
```

```
 link.className = "term-hint";
```

```
 link.href = `#term-${id}`;
```

```
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
```

```
 link.textContent = el.textContent ?? "";
```

```
 el.replaceWith(link);
```

```
 });
```



```

 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style=
<p style="text-align:right;font-size:.72rem;margin:0 0.
 });

 const body = `<nav id="toc" style="background:#0f172a
 rem">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:8
 <ol style="margin:0;padding-left:1.1rem;font-size:.85
</nav>
 ${bodies.join("\n")}`;

 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
 });
 triggerDownload(html, "posobie-vse-temy.html");
}
```

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

ФАЙЛ 77/83: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
/**
```

```
* Аудит покрытия глав пособия: у каких тем нет «объясн
* (блок «Аналогия») и/или 2D-визуализации (интерактивн
*
```

```

 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\s>]/i.test(html),
 image: /<img[\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 });
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
 console.log("| --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(`  • ${r.title} [${r.num}]`));
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
}

```

```

const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
 .replace(/<[^\>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlights).padStart(3),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 8).join(", ")
);
}
console.log("\nГлав без единой подсказки:", bad.length, "шт.");
console.log("Терминов в глоссарии:", GLOSSARY.length, "шт.");

```

```
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```



```
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel));
```

```
/** Порядок разделов в документе. */
```

```
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];
```

```
function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 }
}
```

```
function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { time: true });
 const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ ИСХОДНОГО КОДА)");
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода:            ${humanSize(totalBytes)}`);
 push(`Глав/билетов в пособии:          ${chapters.length}`);
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
 push(HR);
 chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
 });
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
 push(HR);
 let current = null;
 files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 }
 });
}
```

```

 console.log(`        файлов:  ${files.length}`);
 console.log(`        глав:    ${chapters.length}`);
 console.log(`        строк:   ${txt.split("\n").length}`);
 console.log(`        выход:   ${path.relative(ROOT, TXT_PATH)}`);
}
```

```
main();
```

-----  
ФАЙЛ 81/83: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
```

```
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";
```

```
export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
```

```
/** Куда кладём финальные артефакты (эта папка отдаётся клиенту)
export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают)
export const PAGES_DIR = path.join(ROOT, "tmp", "export");
```

```
export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");
```

```
/** Параметры вёрстки страницы (подобраны под A4 @150dpi)
export const PAGE = {
 width: 210, // ширина в мм
 height: 297, // высота в мм
 margin: 10, // отступы в мм
};
```

```

 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick
);
}
```

-----  
ФАЙЛ 82/83: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 3.5 КБ

```

/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu Sans Mono)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize,
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки, чтобы
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/ {4}/g, "\n");
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min(expanded.match(/^ +/).length, 4));
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const cut = rest.length > PAGE.cols ? PAGE.cols : rest.length;
 const part = first ? rest.slice(0, cut) : rest.slice(indent.length, cut + indent.length);
 parts.push(part);
 rest = rest.slice(cut);
 first = false;
 }
 return parts;
}
```

```

 return { txtFile, pngFile, num };
 });

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 10));
let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`      отрендерено ${done}\n`);
 }
 }
}

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length > 0) {
 console.log("Не удалось создать следующие файлы:");
 missing.forEach(f => console.log(f.pngFile));
}

```

```
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бесконечный цикл
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умолчанию 150)"
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-`${{ github.ref }}`
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot]')"
 steps:
 - name: Checkout
 uses: actions/checkout@v4
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v4
 with:
 node-version: "22"
```

```
echo "| Артефакт | Размер |"
echo "| --- | --- |"
echo "| \`public/export/full_code_all_pages
echo "| \`public/export/full_code_all_pages
echo "| PNG-страниц | ${ls tmp/export-pages.
} >> "$GITHUB_STEP_SUMMARY"
```

- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v4  
with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt  
 if-no-files-found: error  
 retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v4  
with:  
 name: full-code-pages-png  
 path: tmp/export-pages/\*.png  
 if-no-files-found: error  
 retention-days: 7
- name: Commit rebuilt export back to the branch  
run: |  
 git config user.name "github-actions[bot]"  
 git config user.email "41898282+github-action  
 git add -- public/export/full\_code\_all\_pages.  
 if git diff --cached --quiet; then  
 echo "Экспорт не изменился — коммит не нужен"  
 exit 0  
 fi  
 git commit -m "chore(export): пересборка full  
 git push origin "HEAD:\${GITHUB\_REF\_NAME}"